

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PRIRODZENÝM JAZYKOM OVLÁDANÝ EDITOR
VO FORME WEBOVEJ APLIKÁCIE
BAKALÁRSKA PRÁCA

2026
TOMÁŠ TELEK

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PRIRODZENÝM JAZYKOM OVLÁDANÝ EDITOR
VO FORME WEBOVEJ APLIKÁCIE
BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra informatiky
Školiteľ: RNDr. Andrej Lúčny, PhD

Bratislava, 2026
Tomáš Telek



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Tomáš Telek
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Prirodzeným jazykom ovládaný editor vo forme webovej aplikácie
Web Application for Editing Controlled by the Natural Language

Anotácia: Veľké jazykové modely umožňujú porozumenie písaným či hovoreným pokynom v miere do nedávna nepredstaviteľnej. Ich služby sú dostupné v Internete, preto je prirodzené využívať ich z webových aplikácií. Konkrétnym príkladom takejto aplikácie je editor, v ktorom všetky operácie robíme pokynmi v prirodzenom jazyku, napríklad: “píš Ema má mamu” (napíše v mieste kurzora “Ema má mamu”) alebo “nahrad’ všetky m za t” (“Eta tá tatu”).

Cieľ: Práca má stanové nasledovné ciele:
- naštudovať a popísať vybranú technológiu vývoja backend-u a frontend-u webovej aplikácie na platforme Java
- naštudovať a popísať ako sa z platformy Java volajú služby veľkého jazykového modelu ako je ChatGPT.
- vyvinúť jednoduchú, ale funkčnú ukážku takejto aplikácie, konkrétne editor ovládaný prirodzeným jazykom (textom, prípadne aj hlasom)

Literatúra: Nicholas S. Williams: Professional Java for Web Applications. Wrox, 2014

Poznámka: Platforma Java, konkrétne prostriedky si môže autor zvoliť

Kľúčové slová: webová aplikácia, Java, editor, veľký jazykový model

Vedúci: RNDr. Andrej Lúčny, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 24.02.2025

Dátum schválenia: 22.09.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Čestné vyhlásenie: Čestne vyhlasujem, že celú bakalársku prácu na tému „Prirodzeným jazykom ovládaný editor vo forme webovej aplikácie“, vrátane všetkých jej príloh a obrázkov, som vypracoval samostatne, a to s použitím literatúry uvedenej v priloženom zozname a nástrojov umelej inteligencie (Gemini, NotebookLM, ChatGPT) na účely konzultácie odborných termínov, štrukturálnej úpravy textu, kontroly gramatickej a syntaktickej správnosti, podpory pri spracovaní vlastných myšlienok a technickej podpory pri programovaní. Nástroje umelej inteligencie som použil v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity.

Abstrakt

Práca sa venuje návrhu a implementácii webového textového editora, ktorý je plne ovládateľný prirodzeným jazykom. Hlavným cieľom bolo vytvoriť prostredie, ktoré používateľovi umožňuje editovať text a generovať nový obsah pomocou hlasových alebo textových príkazov, čím sa eliminuje potreba tradičného manuálneho formátovania. Práca analyzuje možnosti integrácie moderných veľkých jazykových modelov do používateľského rozhrania a navrhuje architektúru založenú na autonómnych softvérových agentoch. Systém je implementovaný ako webová aplikácia v jazyku JavaScript a využíva veľké jazykové modely na spracovanie reči na text a následnú analýzu príkazov. Súčasťou riešenia je mechanizmus automatickej detekcie hlasovej aktivity, ktorý zabezpečuje plynulú interakciu bez manuálneho ovládania záznamu. Funkčnosť navrhnutého riešenia bola overená experimentálnym testovaním v slovenskom jazyku, pričom systém dosiahol úspešnosť 82,5 % pri plnení definovaných úloh. Výsledky práce demonštrujú, že súčasná úroveň umelej inteligencie umožňuje efektívny prechod od klasických grafických rozhraní k inteligentným systémom riadeným prirodzeným jazykom.

Kľúčové slová: textový editor, prirodzený jazyk, veľké jazykové modely, hlasové ovládanie, agentická architektúra.

Abstract

This thesis focuses on the design and implementation of a web-based text editor fully controlled by natural language. The primary objective was to develop an environment that allows users to edit text and generate new content using voice or text commands, thereby eliminating the need for traditional manual formatting. The work analyzes the integration of modern large language models into user interfaces and proposes an architecture based on autonomous software agents. The system is implemented as a web application using JavaScript and utilizes large language models for speech-to-text processing and subsequent command analysis. A key component of the solution is an automated voice activity detection mechanism, which ensures seamless interaction without the need for manual recording control. The functionality of the proposed solution was verified through experimental testing in the Slovak language, where the system achieved a success rate of 82.5% in completing defined tasks. The results demonstrate that the current state of artificial intelligence enables an effective transition from classical graphical user interfaces to intelligent systems driven by natural language.

Keywords: text editor, natural language, large language models, voice control, agent-based architecture.

Obsah

Abstrakt	ix
Úvod	1
Motivácia	1
Cieľ práce	2
1 Prehľad riešenej problematiky	3
1.1 AI asistenti v tradičných editoroch	3
1.2 AI-natívne editory	3
1.3 Systémy pre hlasovú interakciu a diktovanie	4
1.3.1 Diktovanie	4
1.3.2 Inteligentné hlasové diktovanie	5
2 Prostriedky vývoja	7
2.1 Používané technológie a jazyky	7
2.1.1 JavaScript	7
2.1.2 React	8
2.1.3 Štruktúrovaný výstup	8
2.2 Rozhrania pre umelú inteligenciu	9
2.2.1 OpenAI API	9
2.2.2 LangChain	9
2.3 Spracovanie hlasového vstupu	10
2.3.1 Detekcia aktivity reči	10
2.3.2 Model OpenAI Whisper	10
3 Agentic AI	11
3.1 Definícia a charakteristika agenta	11
3.2 Kľúčové komponenty agentických architektúr	13
3.3 Technická implementácia agentov	14
3.3.1 Komunikačné rozhranie a spracovanie správ	15
3.3.2 Rozdiely vo výstupoch: čistý text vs. štruktúrovaný výstup	16

3.3.3	Praktická implementácia: Creative Agent	16
3.4	Multi-agentové systémy	17
3.5	Výzvy a limitácie agentických architektúr	18
4	Analýza a návrh systému	19
4.1	Špecifikácia funkčných požiadaviek	19
4.2	Špecifikácia nefunkčných požiadaviek	20
4.3	Architektonický návrh aplikácie	21
4.4	Návrh používateľského rozhrania	22
4.5	Funkčná analýza a spracovanie pokynov	23
5	Implementácia a vývoj systému	25
5.1	Prvotný prototyp a deterministické riadenie	25
5.1.1	Dátová štruktúra a správa stavu	25
5.1.2	Vizuálne rozhranie: Komponent EditorView	26
5.1.3	Prvotná logika rozhodovania	26
5.1.4	Implementovanie prvých agentov	26
5.2	Integrácia riadiaceho agenta	28
5.2.1	Vynútenie štruktúry a exekúcia	28
5.3	Pridanie ďalších funkcií editora a optimalizácia	29
5.3.1	Creative Agent: Generovanie obsahu a kreativita	29
5.3.2	Rozširovanie funkcií editora a znižovanie latencie	30
5.4	Implementácia hlasového rozhrania a spracovanie audia	31
5.4.1	Manuálne nahrávanie (Whisper API)	31
5.4.2	Streaming a inteligentné ukončovanie	32
5.4.3	Integrácia VAD a Whisper API	33
5.5	Zhrnutie implementačnej fázy	34
5.6	Používateľské rozhranie finálnej aplikácie	34
6	Testovanie a zhodnotenie výsledkov	39
6.1	Verifikácia funkčnosti na vybraných scenároch	39
6.1.1	Testovanie navigácie	39
6.1.2	Testovanie správy dokumentu a vizuálnych vlastností	40
6.1.3	Testovanie inteligentného spracovania a úpravy obsahu	41
6.1.4	Testovanie komplexných a sekvenčných príkazov	41
6.1.5	Zhrnutie verifikačnej fázy	42
6.2	Analýza latencie a odozvy systému	43
6.3	Diskusia o limitoch a presnosti interpretácie príkazov	44
	Záver	45

Literatúra	49
Príloha A	51

Zoznam obrázkov

4.1	Schéma toku informácií zachytávajúca cestu od hlasového vstupu cez VAD a model Whisper až po logiku Router Agentu a finálnu úpravu dokumentu.	22
5.1	Hlavné rozhranie editora s rozdelením na konfiguračnú časť (API kľúč), textovú plochu s číslovanými odsekmi a ovládacie prvky pre hlasový aj textový vstup.	35
5.2	Ukážka vizuálnej odozvy systému. Textové pole zobrazuje prepísaný príkaz z modelu Whisper a stavový riadok indikuje fázu spracovania textu agentmi.	36
5.3	Stav dokumentu po exekúcii príkazu, kde je vidieť úspešná modifikácia konkrétneho odseku pri úplnom zachovaní štruktúry textu.	37

Zoznam tabuliek

3.1	Špecifikácia agenta podľa metodiky PAGE.	12
6.1	Štatistické vyhodnotenie navigačných scenárov	39
6.2	Štatistické vyhodnotenie správy dokumentu a formátovania	40
6.3	Štatistické vyhodnotenie inteligentných funkcií editora	41
6.4	Štatistické vyhodnotenie komplexných a sekvenčných operácií	42
6.5	Namerané hodnoty priemernej latencie podľa kategórií úloh.	43

Zoznam algoritmov

3.1	Implementácia getLCModel	15
3.2	Implementácia callLC	15
3.3	Implementácia Creative Agenta	16
5.1	Implementácia Edit Agenta	27
5.2	Definícia schémy taskScheme	28
5.3	Volanie štruktúrovaného modelu v Router Agentovi	29
5.4	Kompletná definícia taskScheme so všetkými funkciami	30
5.5	Odosielanie audio dát na API Whisper	32
5.6	Inicializácia VAD a spracovanie udalosti ukončenia reči	33

Zoznam skratiek a symbolov

API	Application Programming Interface (Rozhranie pre programovanie aplikácií)
CSS	Cascading Style Sheets (Kaskádové štýly)
HTML	HyperText Markup Language (Značkovací jazyk)
LLM	Large Language Model (Veľký jazykový model)
STT	Speech-to-Text (Prevod reči na text)
VAD	Voice Activity Detection (Detekcia hlasovej aktivity)
AI	Artificial Intelligence (Umelá inteligencia)

Úvod

Nástup **veľkých jazykových modelov** (angl. Large Language Models – LLM) otvára dvere k úplne novej generácii aplikácií. V týchto systémoch umelá inteligencia netvorí len doplnok, ale samotné jadro rozhrania. Umožňuje spracovávať príkazy v prirodzenom jazyku s mierou porozumenia, ktorá bola doteraz nepredstaviteľná. Napriek tomuto pokroku väčšina súčasných textových editorov stále používa manuálne písanie príkazov, čo naruša plynulosť tvorivého procesu.

Táto bakalárska práca sa zameriava na odstránenie tejto bariéry prostredníctvom vývoja webového editora, ktorý je plne ovládaný prirodzeným jazykom, primárne pomocou hlasu. Cieľom je vytvoriť systém, kde používateľ nemusí text manuálne označovať ani formátovať pomocou grafického rozhrania. Namiesto toho komunikuje s editorom ako s digitálnym spoluautorom, ktorý rozumie kontextu dokumentu a dokáže autonómne vykonávať editačné, navigačné či kreatívne úlohy.

Práca opisuje návrh a implementáciu systému, ktorý kombinuje moderné webové technológie s pokročilými modelmi umelej inteligencie od spoločnosti OpenAI. Kľúčovým prvkom riešenia je využitie architektúry autonómnych agentov, ktorá transformuje statický jazykový model na aktívneho agenta schopného modifikovať prostredie editora.

Motivácia

Motiváciou tejto práce je zefektívnenie interakcie medzi človekom a počítačom pri práci s textom. Hoci je diktovanie prirodzeným a rýchlym spôsobom zadávania informácií, bežné systémy diktovania fungujú primárne na princípe pasívneho prepisu reči na text bez hlbšieho pochopenia obsahu. Tento prístup vytvára bariéru v tvorivom procese – ak používateľ urobí chybu alebo potrebuje zmeniť štruktúru textu, musí prestať rozprávať a opraviť ju manuálne pomocou klávesnice alebo myši.

Cieľ práce

Cieľom tejto práce je navrhnúť a vyvinúť funkčnú ukážku webovej aplikácie, editora ovládaného prirodzeným jazykom. Namiesto klasického klikania na ikony a manuálneho formátovania chceme dosiahnuť stav, kedy sa aplikácia stane digitálnym partnerom, ktorému stačí povedať, čo má urobiť.

Kľúčovou vlastnosťou systému je schopnosť inteligentne rozlíšiť, kedy sa len dopĺňa obsah a kedy sa dáva pokyn na zmenu dokumentu. Celá interakcia by mala prebiehať plynulo a bez nutnosti neustáleho stláčania tlačidiel. Výsledkom bude nástroj, ktorý nielen zapisuje slová, ale aktívne rozumie dokumentu a dokáže v ňom samostatne navigovať a vykonávať úpravy na základe jednoduchých hlasových povelov v prirodzenom jazyku.

1 Prehľad riešenej problematiky

Pri návrhu nášho systému vychádzame z už existujúcich riešení, preto najprv uvádzame prehľad bežne používaných nástrojov.

Rozoberieme si rozdiely medzi klasickými editormi a novou generáciou aplikácií, ktoré vznikli priamo s cieľom využívať možnosti umelej inteligencie, takéto nástroje budeme nazývať AI-natívne editory. V záverečnej časti kapitoly sa zameriame na súčasné možnosti hlasového ovládania textových editorov a identifikujeme ich najväčšie nedostatky, ktoré sa snažíme v rámci tejto práce odstrániť.

1.1 AI asistenti v tradičných editoroch

Pod pojmom **tradičné editory** rozumieme programy, ktoré vznikli ešte pred nástupom umelej inteligencie a ich hlavný spôsob ovládania je pomocou *klávesnice a myši*. Ide o nástroje, ktoré používame v každodennom živote, ako napríklad *Microsoft Word* alebo *Google Docs*. Tieto editory sa pôvodne sústredili na písanie textu, jeho vizuálnu úpravu a formátovanie.

V posledných rokoch do nich vývojári začali pridávať funkcie **umelej inteligencie**, väčšinou vo forme pomocníkov, ako je napríklad *Microsoft Copilot*. Tieto modely sú integrované do bočných panelov alebo špeciálnych okien, kde očakávajú vstupy od používateľa. Ich hlavnou úlohou je pomôcť s písaním, opravou gramatiky alebo zhrnutím dlhého textu.[18]

Hoci sú títo asistenti veľmi užitoční, ich ovládanie je stále pevne naviazané na *manuálnu prácu*. Používateľ musí väčšinou text ručne označiť myšou a následne *AI* asistentovi napísať, čo má s textom urobiť. Práve táto potreba neustáleho prepínania medzi písaním textu a ovládaním *AI* nástrojov je jednou z bariér, ktoré sa snažíme v našej práci prekonať.

1.2 AI-natívne editory

AI-natívne editory boli od začiatku navrhované s cieľom naplno využívať umelú inteligenciu. *AI* v nich nie je len doplnkom v bočnom menu, ale tvorí samotné jadro

aplikácie. Typickým príkladom je nástroj *Notion AI* [10].

Tento editor funguje na prvý pohľad ako bežný textový procesor, do ktorého používateľ píše text pomocou klávesnice. Skutočná zmena však nastáva v momente, keď sa využije sila umelej inteligencie priamo tam, kde sa v texte nachádzate. Stačí na mieste kurzora napísať príkaz „/ai“ a editor vám okamžite ponúkne menu s funkciami AI. Za týmto príkazom potom stačí v prirodzenej reči zadať, čo potrebujete. V tej chvíli sa editor prestáva správať ako papier, na ktorý píšete, a začína fungovať ako digitálny spoluautor, ktorý presne vie, o čom sa písalo doteraz.

Systém totiž nečíta len váš posledný príkaz, ale analyzuje celý text, ktorý už na stránke máte. Vďaka tomu dokáže v priebehu pár sekúnd vytvoriť zložité tabuľky, zoznamy úloh alebo dopísať celé kapitoly, ktoré štýlom dokonale nadväzujú na predchádzajúci text. Okrem vytvárania nového obsahu vie pracovať aj s tým, čo už je napísané. Dokáže napríklad zmeniť tón e-mailu z prísne profesionálneho na priateľský, dokáže z dlhého textu vytvoriť stručný súhrn najpodstatnejších myšlienok alebo opraviť stylistické chyby.

Najväčšou výhodou týchto editorov je, že používateľ nemusí text nikam kopírovať ani manuálne formátovať výsledky, pretože *AI* pracuje priamo s **prvkami dokumentu**. Aj pri tejto vysokej miere inteligencie však narážame na problém. Používateľ musí svoje príkazy ručne vyťukávať, čo prerušuje plynulosť práce. Práve túto bariéru chceme odstrániť tým, že silu takéhoto inteligentného editora sprístupníme prostredníctvom **hlasových povelov**. [4]

1.3 Systémy pre hlasovú interakciu a diktovanie

V súčasnosti existuje niekoľko spôsobov, ako môžeme **hlas využiť** pri práci s textom. Tieto technológie prešli v posledných rokoch veľkým vývojom. Od jednoduchého *prepísovania reči* až po systémy, ktoré rozumejú *zložitým príkazom*. V tejto časti si rozoberieme, ako fungujú moderné hlasové editory a prečo ich súčasné nastavenie stále vyžaduje veľa práce s myšou a klávesnicou.

1.3.1 Diktovanie

Najbežnejším spôsobom, ako dnes využiť hlas pri písaní, je klasické diktovanie. Túto funkciu nájdeme v takmer každom modernom zariadení, od textových editorov na smartfónoch až po profesionálne kancelárske programy. Princíp fungovania je založený na premene hovoreného slova na text, čo je podľa moderných štúdií až trikrát rýchlejšie než klasické písanie na klávesnici.

Hoci je táto technológia už veľmi presná a dokáže ušetriť čas pri písaní dlhých textov, má jedno zásadné obmedzenie. Systém totiž textu vôbec nerozumie. Funguje len

ako „písací stroj“ , ktorý namiesto úderov do klávesnice reaguje na zvuk. Ak používateľ urobí chybu, povie niečo nesprávne alebo sa rozhodne zmeniť poradie viet, diktovanie mu v tom nepomôže.

V takom prípade musí používateľ zastaviť nahrávanie a chybu opraviť ručne. Práve táto neschopnosť systému vykonávať opravy a úpravy textu priamo hlasom je dôvodom, prečo sa diktovanie doteraz nestalo plnohodnotnou náhradou za tradičné písanie, ale slúži skôr len ako jeho doplnok. [15]

1.3.2 Inteligentné hlasové diktovanie

Moderné platformy, ako sú *Voicy* [19] alebo *Wispr Flow* [22], posúvajú klasické diktovanie na úplne novú úroveň vďaka využitiu veľkých jazykových modelov. Ich hlavnou výhodou je, že sa už nemusíte sústreďovať na každé jedno vyslovené slovo či dokonalú výslovnosť.

Ak sa pri diktovaní prerieknete, zaseknete alebo sa uprostred vety opravíte, systém do dokumentu nezapíše vaše chyby, ale len tú správnu, finálnu podobu myšlienky. Okrem toho im môžete zadávať aj priame príkazy. Napríklad ich požiadať, aby práve nadiktovaný text skrátili alebo ho prepísali do profesionálnejšieho tónu.

Aj tieto nástroje však majú dva veľké nedostatky, ktoré človeka pri práci brzdia. Prvým je to, že používateľ nemôže len tak rozprávať a sledovať, ako sa text píše. Musí najprv stlačiť tlačidlo, povedať všetko, čo chce, aby sa napísalo, potom nahrávanie zastaviť a až vtedy sa text v dokumente objaví. To je neprirodzené, pretože človek nevidí výsledok okamžite a musí stále myslieť na to, kedy má nahrávanie zapnúť a vypnúť.

Druhým problémom je, že ak chcete niečo v texte upraviť, stále k tomu potrebujete ruky. Systém nevie sám od seba, ktorú vetu chcete zmeniť. Musíte teda vziať do ruky myš, daný kus textu ručne označiť a až potom mu hlasom povedať, čo s ním má urobiť. Človek tak neustále preskakuje medzi rozprávaním a klikaním myšou, namiesto toho, aby sa plne sústredil na to, čo chce napísať.

2 Prostriedky vývoja

Kapitola predstavuje kľúčové technológie, nástroje a programovacie rozhrania, ktoré sa využili pri realizácii textového editora. Pri výbere týchto nástrojov bola snaha vytvoriť modernú webovú aplikáciu, ktorá dokáže plynulo spracovávať hlasové vstupy a v reálnom čase komunikovať s modelmi umelej inteligencie. Dôraz sa kládol najmä na rýchlosť odozvy, bezpečnosť dát a jednoduchú rozšíriteľnosť systému o nové funkcie.

2.1 Použité technológie a jazyky

Realizácia moderného webového editora si vyžaduje kombináciu nástrojov, ktoré umožňujú efektívnu prácu s dátami a plynulé zobrazovanie zmien. Hlavným cieľom bolo, aby editor nefungoval ako jeden celok, ale ako súbor nezávislých prvkov. Tento prístup je kľúčový najmä pri komunikácii s modelmi umelej inteligencie. Ak by systém pri každom hlasovom povelí spracovával celý dokument naraz, dochádzalo by k zbytočnému plytvaniu výpočtovými prostriedkami a predlžovaniu odozvy. Každý odsek má tak v systéme vlastné miesto, čo zjednodušuje jeho ovládanie hlasom a zefektívňuje prácu agentov.

2.1.1 JavaScript

JavaScript bol zvolený ako základný programovací jazyk aplikácie, pretože umožňuje vytvárať plynulé používateľské rozhrania. Vďaka asynchrónnej komunikácii môže používateľ nerušene pracovať s editorom, zatiaľ čo systém na pozadí spracováva požiadavky. Využíva udalosťami riadený programovací model, v ktorom systém plynule reaguje na vstupy používateľa, v tomto prípade na hlasové povely transformované na text.

V rámci implementácie sa jazyk využíva najmä na definovanie riadiacej logiky a správu dátových štruktúr, ktoré reprezentujú dokument ako pole dátových prvkov. Kľúčovou vlastnosťou je tu práve schopnosť asynchrónneho spracovania požiadaviek. Kým aplikácia čaká na odpoveď z externého jazykového modelu, používateľské rozhranie zostáva responzívne a plynulé. To umožňuje, aby proces rozpoznávania hlasu a následná úprava dokumentu prebiehali na pozadí bez narušenia používateľského zážitku.[5]

2.1.2 React

Pre vývoj používateľského rozhrania bola zvolená knižnica *React* [9]. Hlavným dôvodom bol spôsob, akým dokáže pracovať s **dynamickým obsahom**. Na rozdiel od klasických prístupov, kde sa pri každej zmene musí prekresľovať celá stránka, *React* umožňuje vnímať dokument ako sústavu nezávislých komponentov. Tento prístup je kľúčový pre efektívnu komunikáciu s AI, pretože systému dovoľuje manipulovať s textom na úrovni jednotlivých odsekov bez ovplyvnenia zvyšku dokumentu.

Kľúčovou technológiou *Reactu* je tzv. **virtuálny DOM** [8], ktorý zabezpečuje, že aplikácia nezaťažuje prehliadač zmenami v celom rozhraní, ale aktualizuje iba tie časti, ktoré sa reálne zmenili. [1]

Rozdelenie dokumentu na samostatne riadené jednotky prináša zásadnú výhodu v oblasti optimalizácie nákladov a presnosti. Tým, že do modelu OpenAI nie je potrebné odosielať celý dokument, ale iba konkrétny fragment textu, dochádza k výraznej úspore tokenov. Model sa vďaka tomu dokáže plne sústrediť na jeden odsek, čím minimalizuje riziko, že by pri spracovaní dlhého textu prehliadol dôležitý detail alebo spôsobil nežiaducu halucináciu. Menší objem prenášaných dát zároveň priamo zvyšuje rýchlosť odozvy, pretože agent dokáže vygenerovať a vrátiť spracovaný príkaz oveľa rýchlejšie než pri analýze rozsiahlych blokov textu.

2.1.3 Štruktúrovaný výstup

V tejto práci nie je komunikácia s modelom postavená na spracovaní voľného textu, ale na technológii *Structured Output*. Tento prístup slúži ako kľúčový komunikačný most medzi výstupom z umelej inteligencie a riadiacou logikou aplikácie, pričom zabezpečuje transformáciu povelu v prirodzenej reči na presne definovaný, strojovo spracovateľný objekt.

Hlavným pilierom tejto architektúry je knižnica *zod* [7], ktorá umožňuje definovať striktnú schému (**taskScheme**), ktorej štruktúra je znázornená v ukážke 5.2. Táto schéma určuje presné hranice, v ktorých sa model môže pohybovať. Definuje typy očakávaných parametrov a ich podrobný opis. Táto technológia sa primárne využíva pri implementácii **Router Agent**a, kde model na základe používateľovho príkazu vyberá z vopred definovaných kategórií úloh (napr. *edit*, *creative*, *navigate*, *font*, *structure* či *undo*). Využitím tejto schémy je *LLM* model prinútený zaradiť požiadavku do jednej z definovaných možností, čím sa eliminuje riziko nepredvídateľného správania.

Využitie schémy *zod* v kombinácii so štruktúrovaným výstupom modelu teda výrazne zjednodušuje implementáciu kontrolných mechanizmov. Zabezpečuje, že editor vykoná len tie akcie, ktoré sú jasne definované a zodpovedajú možnostiam, ktoré mu **Router Agent** pridelil. [7]

2.2 Rozhrania pre umelú inteligenciu

Pre fungovanie prirodzeného ovládania editora je nevyhnutné prepojiť používateľské rozhranie s externými službami, ktoré zabezpečujú spracovanie prirodzeného jazyka a interpretáciu hlasu. V tejto časti sú opísané kľúčové API rozhrania a frameworky, ktoré tvoria intelektuálnu vrstvu aplikácie.

2.2.1 OpenAI API

Ako hlavný generatívny model bol zvolený systém od spoločnosti OpenAI[12], konkrétne model **gpt-4o**. Výber tohto modelu bol podmienený najmä optimálnym pomerom medzi **nízkou latenciou spracovania** a **vysokou kvalitou generovaných odpovedí**. V kontexte hlasom ovládaného editora je rýchlosť odozvy kritická pre zachovanie plynulosti interakcie, zatiaľ čo pokročilé kognitívne schopnosti modelu zabezpečujú presnú interpretáciu aj komplexných či nejednoznačných hlasových povelov. Model prijíma textovú požiadavku spolu s kontextom dokumentu a na základe systémových inštrukcií transformuje prirodzený jazyk na exekutívne príkazy spracovateľnej aplikáciou.

2.2.2 LangChain

Pre efektívne riadenie komunikácie medzi jazykovým modelom a komponentmi editora bol použitý framework *LangChain* [6]. V projekte je od základov navrhnutý tak, aby umožňoval implementáciu agentov.

Hlavným prínosom *LangChainu* je schopnosť rozširovať základné schopnosti jazykového modelu o prácu s externými dátami a nástrojmi, čím premieňa statický model na aktívneho agenta. Framework v rámci jednej logickej operácie spája rôzne zdroje dát, vďaka čomu model v každom dopyte dostáva ucelený balík informácií: identitu systému s jasnými pravidlami, aktuálny stav dokumentu a samotný hlasový povel od používateľa. *LangChain* následne dohliada na to, aby odpoveď modelu nebola len bežnou konverzáciou, ale konkrétnym a správnym výsledkom.

Z hľadiska praktickej realizácie boli v rámci frameworku vytvorené kľúčové funkcie pre inicializáciu modelu (`getLCModel`) a transformáciu správ do objektového formátu (`callLC`). Detailná implementácia týchto funkcií, vrátane ukážok zdrojového kódu, je popísaná v kapitole venovanej technickej implementácii agentov (viď na strane 15).

Bez využitia tohto frameworku by bolo programovanie interakcií medzi LLM a aplikáciou zložitejšie. *LangChain* poskytuje potrebné abstrakcie, ktoré umožňujú LLM vystúpiť z rámca bežného generovania textu a stať sa funkčnou súčasťou softvérového systému, ktorá dokáže cielene modifikovať dáta.

2.3 Spracovanie hlasového vstupu

Pre zabezpečenie funkčnosti editora bez nutnosti manuálnych zásahov je proces snímania hlasu navrhnutý ako plynulý a prirodzený systém. Celý proces je rozdelený do dvoch kľúčových fáz: lokálnej detekcie aktivity reči (*Voice Activity Detection* – VAD) [24] a následného prepisu audia na text pomocou modelu *Whisper* [11].

2.3.1 Detekcia aktivity reči

Kľúčovou technickou výzvou pri návrhu systému je zabezpečenie prevádzky bez potreby manuálneho spúšťania a ukončovania nahrávania. Na tento účel je zvolená technológia VAD, ktorá umožňuje inteligentné riadenie audio vstupu.

V rámci systému je využívaná knižnica `ricky0123/vad-web`[14], ktorá do prostredia webového prehliadača prináša pokročilý model *Silero VAD*[17]. Podstatným rozdielom oproti bežným detektorom ticha je, že tento nástroj využíva integrovanú neurónovú sieť vytrénovanú špeciálne na identifikáciu ľudského hlasu.

Proces teda nefunguje len na základe sledovania hlasitosti, ale prostredníctvom analýzy zvukových vĺn v reálnom čase rozpoznáva charakteristické črty reči. Vďaka tomuto zapojeniu umelej inteligencie už na lokálnej úrovni prehliadača je systém schopný odlíšiť hlas používateľa od okolitého šumu či iných rušivých vplyvov. Knižnica následne umožňuje definovať funkcie, ktoré po rozpoznaní konca prehovoru automaticky spracujú nahrávku (viď ukážku 5.6). [14]

2.3.2 Model OpenAI Whisper

Na transformáciu hovoreného slova na text bol zvolený model *Whisper*, taktiež od spoločnosti OpenAI. Na rozdiel od natívnych riešení v prehliadačoch, ktoré často majú problém s presnosťou a spracovaním slovenského jazyka, *Whisper* poskytuje rozpoznávanie reči na vysokej úrovni.

Audio signál zachytený z mikrofónu je odosielaný na serverové spracovanie, kde je pomocou hlbokých neurónových sietí pretransformovaný na textový reťazec. Model vykazuje vysokú odolnosť voči šumu, čo umožňuje používateľovi diktovať príkazy aj v menej ideálnych podmienkach. Významnou výhodou je schopnosť modelu korektne identifikovať odbornú terminológiu a technické výrazy, ktoré by bežné diktovacie nástroje mohli zameniť za foneticky podobné slová z bežnej reči.

Pri výbere tohto modelu bola uprednostnená kvalita prepisu pred minimalizáciou latencie. Pre správne fungovanie AI agentov je tento aspekt kritický. Ak by bol prepis hlasového povelu nepresný, agent by dostal chybný vstup a vykonal by nesprávnu editačnú akciu. [13]

3 Agentic AI

Aby textový editor dokázal plniť hlasové príkazy bez toho, aby sme museli klikať myšou, nestačí využívať veľký jazykový model len ako generátor textu. Je potrebné aplikovať prístup **Agentic AI** [21]. Pod týmto pojmom rozumieme techniky, ktoré rozširujú základné schopnosti LLM o schopnosť získavať informácie z prostredia aplikácie (agent vie, čo presne je v dokumente) a následne vykonávať konkrétne akcie na základe týchto údajov (agent dokáže sám prepísať alebo sformátovať text, ktorý sa zobrazí v editore). Zatiaľ čo bežný LLM len odpovedá na otázky ako v čete, agent v tomto editore dokáže textu nielen porozumieť, ale ho aj sám reálne meniť.

3.1 Definícia a charakteristika agenta

Bežné funkcie umelej inteligencie, ktoré dnes poznáme z textových editorov, fungujú skôr ako šikovné doplnky. Sú to nástroje, ktoré vedia automaticky doplniť slovo alebo opraviť gramatickú chybu, ale stále len čakajú na váš priamy pokyn. Pri *Agentic AI* sa však posúvame od pojmu „nástroj“ k pojmu „agent“.

Zásadný rozdiel nespočíva len v kvalite odpovedí, ale v spôsobe, akým model pracuje s okolitým systémom. **Bežný program** len pasívne vykonáva presne zadané inštrukcie a ak mu nie je dodaný kontext, nedokáže pracovať s aktuálnym stavom dokumentu. **Agentovi** však stačí definovať cieľový stav a on na základe aktuálnych údajov z dokumentu sám naplánuje a vykoná potrebné kroky.

Tento rozdiel najlepšie uvidíme na zložitejšom príkaze, ako je napríklad: *„Vynechaj v texte informácie, ktoré nie sú dôležité.“* Bežný editor by na takýto pokyn nevedel zareagovať, pretože netuší, ktoré slová sú dôležité a ktoré nie. **Agent** si však text najprv „prečíta“, pochopí jeho hlavnú myšlienku a následne sám identifikuje a vymaže tie časti, ktoré sú navyše. Používateľ teda nemusí v texte nič ručne označovať ani mazať, agent to urobí za neho. Pre lepšie pochopenie si môžeme fungovanie takéhoto agenta predstaviť cez tri základné schopnosti:

- **Senzory(zisťovanie informácií):** Aby agent vedel, čo má robiť, potrebuje svoje „zmysly“. V našom systéme sú to mikrofón a aktuálny obsah dokumentu. Agent

teda nevníma len hlasový príkaz, ale „vidí“ aj celý kontext toho, čo je v editore napísané. Vďaka tomu presne vie, na ktorú časť textu príkaz smeruje.

- **Agentický program:** Toto je jadro celého systému. Na rozdiel od bežných programov, ktoré fungujú podľa vopred daných pravidiel, náš agent využíva veľký jazykový model na to, aby pochopil skutočný zámer. Ak mu napríklad povie: „Prepíš tento odsek tak, aby znel viac profesionálne“, agent musí urobiť niekoľko zložitých rozhodnutí. Musí identifikovať, ktoré slová sú príliš neformálne, pochopiť hlavnú myšlienku textu a následne ho celý preštylizovať tak, aby si zachoval význam, ale zmenil pôsobivý dojem. Agent sa teda nesústredí len na jednotlivé slová, ale na celkový kontext a následne sa rozhodne, ako text upraví.
- **Efektory (vykonávanie akcií):** Aby agent mohol vykonať zmeny, má k dispozícii sadu nástrojov. Sú to programové funkcie editora, ktoré mu umožňujú písať, mazať alebo formátovať text. Agent sa sám rozhodne, ktorý nástroj v danom momente použije, aby splnil cieľ.

Pre presný opis toho, ako náš agent v systéme funguje, využívame metodiku **PAGE** (z angl. Percepts, Actions, Goals, Environment). Táto metodika nám pomáha jasne definovať hranice systému. Určuje, z čoho agent čerpá informácie, aké má možnosti a čo je jeho hlavnou úlohou. [16]

Zložka	Popis v našom systéme
Vnemy (Percepts)	Všetko, čo agent v danom momente prijíma – teda prepis hlasu na text a aktuálny obsah dokumentu, s ktorým pracuje.
Akcie (Actions)	Konkrétne kroky, ktoré môže agent v editore vykonať, ako napríklad napísanie novej vety, vymazanie odseku alebo pohyb v texte.
Ciele (Goals)	Hlavný zámer, ktorý má agent dosiahnuť – teda úspešne vykonať hlasový príkaz bez potreby manuálneho ovládania.
Prostredie (Environment)	Digitálny priestor textového editora, v ktorom sa celý proces odohráva.

Tabuľka 3.1: Špecifikácia agenta podľa metodiky PAGE.

3.2 Kľúčové komponenty agentických architektúr

Zatiaľ čo v predchádzajúcej časti sme si agenta definovali z pohľadu jeho vonkajšej interakcie s prostredím (vnímanie a akcie), táto sekcia sa zameriava na jeho vnútornú logickú štruktúru. Aby totiž agent dokázal premeniť zachytené vnemy na správne a logické akcie, potrebuje pevnú **agentickú architektúru**.

Ide o súbor vnútorných pravidiel a mechanizmov, ktoré určujú, ako má model spracovávať informácie a vykonávať zmeny v dokumente. V praxi nám toto umožňuje premeniť *jazykový model* na **autonómnu jednotku**, ktorá sa dokáže sama rozhodnúť, kedy má text opraviť, kedy sa v ňom má presunúť inam a ktorý konkrétny nástroj má použiť.

Moderný **agentický systém** si môžeme predstaviť ako sústavu štyroch kľúčových modulov, z ktorých každý zodpovedá za inú časť jeho logiky:

- **Profílovanie:** Tento modul určuje, akú rolu má agent v systéme zohrávať. V našom riešení to dosahujeme pomocou systémových inštrukcií (*System Prompts*). Ide o špeciálny druh textového zadania pre jazykový model, ktoré nie je určené pre používateľa, ale slúži ako „nastavenie pravidiel“ pre samotnú AI. V týchto inštrukciách modelu presne zdefinujeme jeho identitu a odborné zameranie ešte predtým, než začne spracovávať konkrétne príkazy.

Napríklad **Edit Agent** má v systémovej inštrukcii jasne určené, že jeho jedinou úlohou je upravovať text podľa požiadaviek používateľa. Keďže má model takto vopred vymedzené hranice, sústreďí sa len na jednu konkrétnu činnosť. Vďaka tomu robí systém oveľa menej chýb a nepokúša sa vykonávať operácie, na ktoré nie je určený.

- **Plánovanie:** Modul zabezpečuje, aby agent nezačal plniť úlohu bezhlavo, ale aby si ju najprv rozdelil na menšie a logické kroky. V našom systéme túto úlohu plní *Router Agent*, ktorý funguje ako inteligentný rozhodovací uzol. Keď dostane príkaz, nevykoná hneď prvú náhodnú operáciu, ale využije metódu *Chain of Thought* (reťazec myšlienok). Ide o techniku, pri ktorej model vygeneruje logický postup krokov, ktorými sa dopracuje k výsledku.

Napríklad, ak používateľ povie: „Oprav chybu v predchádzajúcom odseku“, agent si v rámci jedného cyklu naplánuje postupnosť dvoch krokov: najprv sa musí v dokumente presunúť na správne miesto (navigácia) a až potom môže vykonať samotnú opravu textu (editácia). Vďaka tomuto plánovaniu dokáže systém riešiť aj zložitejšie zadania bez toho, aby sa v procese stratil.

- **Pamäť:** Pamäť umožňuje agentovi udržiavať informácie o stave editora. V rámci navrhovaného systému rozlišujeme dva typy:

- *Krátkodobá pamäť*: Slúži na spracovanie konkrétnej úlohy v danom momente. Predstavuje aktuálny výsek textu (napríklad jeden odsek), ktorý editor pošle jazykovému modelu spolu s príkazom používateľa. Táto pamäť je dočasná, slúži len na to, aby AI „videla“ text, ktorý má práve upraviť, a po vykonaní zmeny sa uvoľní pre ďalšiu úlohu.
 - *Dlhodobá pamäť*: Drží informáciu o celkovom stave dokumentu a o tom, kde sa v ňom práve nachádza. Vďaka nej agent rozumie aj relatívnym príkazom, ako napríklad „oprav chybu v predchádzajúcom odseku“, pretože vie, na akom odseku sme skončili. Súčasťou tejto pamäte je aj história zmien (*Undo*), ktorá funguje ako bezpečnostná poistka. Umožňuje systému zapamätať si predchádzajúce verzie textu a v prípade potreby sa k nim vrátiť.
- **Akčný modul** : Toto je časť systému, ktorá robí hlavný rozdiel medzi bežným chatbotom a skutočným agentom. Zatiaľ čo chatbot vám len odpovie textom, akčný modul umožňuje agentovi reálne zasahovať do dokumentu. V našom projekte na to využívame metódu volania naprogramovaných funkcií.

Táto schopnosť dovoľuje jazykovému modelu rozpoznať, kedy je potrebné vykonať konkrétnu akciu (napríklad zmazať vetu alebo sa presunúť na iný odsek). Namiesto písania dlhých vysvetlení model vygeneruje krátky technický pokyn, ktorému program editora okamžite rozumie. Tento pokyn slúži ako priamy povel pre funkcie editora, ktoré následne vykonajú požadovanú zmenu bez potreby ďalšieho zásahu používateľa.[21]

3.3 Technická implementácia agentov

Základom implementácie je prostredie *React*, v ktorom agenti operujú nad dynamickým stavom dokumentu. Keďže každý agent potrebuje pre svoje rozhodovanie poznať aktuálny kontext, dostáva pri volaní parametre, ako sú samotný text odseku a index aktuálneho odseku.

Na prepojenie aplikácie s inteligenciou modelu **GPT-4o** nevyužívame priame volania API, ale framework **LangChain**. Vďaka nemu môžeme používať abstrakčné nástroje, ktoré automaticky formátujú dáta do požadovanej štruktúry a spravujú komunikačné rozhranie s *LLM*. Celá logika je implementovaná v jazyku *TypeScript*, čo nám umožňuje striktné definovať dátové typy a predchádzať chybám pri spracovaní komplexných stavov editora.

3.3.1 Komunikačné rozhranie a spracovanie správ

Aby sme nemuseli pri každom hlasovom príkaze manuálne definovať identitu a pravidlá správania modelu, vytvorili sme dve hlavné funkcie. Tie fungujú ako komunikačné rozhranie medzi aplikačnou logikou a umelou inteligenciou:

- `getLCModel(apiKey)`: Táto funkcia zabezpečuje inicializáciu samotného modelu (konkrétne *GPT-4o*) a definuje jeho základné prevádzkové parametre. Jej hlavnou úlohou je vytvoriť stabilné spojenie s rozhraním *OpenAI* a pripraviť model na prijímanie inštrukcií v rámci rámca *LangChain*.

```
1  export function getLCModel(apiKey: string, temperature = 0) {  
2    if (!apiKey) {  
3      throw new Error("Missing API key in getLCModel()");  
4    }  
5    return new ChatOpenAI({  
6      apiKey,  
7      model: "gpt-4o",  
8      temperature: temperature,  
9    });  
10 }
```

Algoritmus 3.1: Implementácia `getLCModel`

- `callLC(model, messages)`: Táto funkcia transformuje surové dáta z editora do štruktúrovaného formátu, ktorému *OpenAI* rozumie. Informácie rozdeľuje do troch základných logických kategórií:
 1. **SystemMessage**: Definuje pravidlá, ktoré má *LLM* dodržať, a identitu jednotlivých agentov.
 2. **AIMessage**: Reprezentuje predchádzajúce odpovede modelu, čo umožňuje systému pracovať s históriou komunikácie.
 3. **HumanMessage**: Obsahuje konkrétne vstupy a povelý od používateľa.

```
1  export async function callLC(model: ChatOpenAI, messages: any[]) {  
2    const converted = messages.map((m) => {  
3      if (m.role === "system") return new SystemMessage(m.content);  
4      if (m.role === "assistant") return new AIMessage(m.content);  
5      return new HumanMessage(m.content);  
6    });  
7  
8    const response = await model.invoke(converted);  
9    return response.content as string;  
10 }
```

Algoritmus 3.2: Implementácia `callLC`

3.3.2 Rozdiely vo výstupoch: čistý text vs. štruktúrovaný výstup

Pri návrhu architektúry sme rozlišovali, v akom formáte majú agenti vracať dáta, aby bol systém čo najefektívnejší:

Špecializovaní agenti (Creative/Edit):

Pri týchto agentoch využívame funkciu `callLC` na získanie priamej textovej odpovede. Keďže je ich úlohou vrátiť len upravený alebo vygenerovaný obsah odseku, bolo by zbytočné, aby model vracal štruktúrovaný výstup. Výsledný čistý text sa následne priamo zapracuje do stavu dokumentu.

Router Agent:

Tento agent plní funkciu plánovača celého systému. Na model aplikujeme metódu `.withStructuredOutput(taskSchema)`, ktorá predstavuje kľúčový mechanizmus pre stabilitu aplikácie. Táto metóda núti jazykový model, aby vygeneroval *štruktúrovaný výstup* namiesto prirodzeného textu. Samotná `taskSchema` je definovaná pomocou knižnice *zod* a slúži ako striktný predpis, ktorý určuje presný zoznam povolených hodnôt (viď ukážka 5.2). V praxi tak model dostáva jasne vymedzené možnosti, z ktorých si má vybrať (napr. *edit* alebo *navigate*), čím sa úplne eliminuje riziko, že *AI* vygeneruje neplatný príkaz, ktorý by riadiaca logika aplikácie nedokázala spracovať.

3.3.3 Praktická implementácia: Creative Agent

Pre lepšiu ilustráciu fungovania špecializovaných agentov je uvedená implementácia kreatívneho agenta. Agent demonštruje využitie čistého textového výstupu bez nutnosti štruktúrovania, pričom sa spolieha na presne definovaný *System Prompt*. *Creative Agent* slúži na generovanie a rozširovanie textového obsahu v danom odseku. Jeho hlavnou úlohou je analyzovať kontext aktuálneho a susedných odsekov a na základe hlasového povelu plynule nadviazať na rozpísanú myšlienku, pričom striktne zachováva štýl, tón a slovnú zásobu autora.

```
1 export async function runCreativeAgent(opts: { ... }) {
2
3   const { apiKey, paragraph, command, language } = opts;
4   const system = 'You are a Creative Writing Assistant.
5     Your goal is to generate or expand text based on the users prompt and the existing context.
6
7     RULES:
8     1. CONTINUATION: If the user asks to "continue" or "write more", analyze the current paragraph
9       and generate a seamless, natural addition.
10    2. CREATION: If the user asks to "generate" or "write a story about", create new, engaging
11       content from scratch.
12    3. TONE: Match the style, vocabulary, and emotional tone of the existing paragraph.
```



```

11 4. OUTPUT: Return ONLY the newly generated text. Do not include any meta-comments.';
12
13 const messages = [
14 { role: "system", content: system },
15 { role: "user",      content: 'CURRENT PARAGRAPH: ""${paragraph}"" USER COMMAND: ${command}' }];
16
17 const model = getLCModel(apiKey, 0.8);
18 return (await callLC(model, messages)).trim();
19 }

```

Algoritmus 3.3: Implementácia Creative Agenta

Uvedená implementácia v praxi potvrdzuje princípy opísané v predchádzajúcich častiach, najmä využitie *System Promptu* na striktné riadenie správania modelu. Agent tak predstavuje časť systému, ktorý dokáže do odseku vygenerovať text na základe hlasového povelu.

3.4 Multi-agentové systémy

Pri riešení komplexných úloh v prostredí textového editora narážajú systémy postavené na jednom univerzálnom modeli na svoje limity. Často ide o problémy s udržaním širšieho kontextu alebo presným vykonávaním detailných inštrukcií. Efektívnym prístupom k prekonaniu týchto bariér sú **multi-agentové systémy**[23], ktoré namiesto jednej komplexnej inštrukcie využívajú sadu viacerých špecializovaných komponentov.

Hlavná výhoda tohto prístupu spočíva v rozdelení logiky medzi viacerých agentov s konkrétnym zameraním. Tento princíp špecializácie je aplikovaný aj v navrhovanom hlasovom editore, kde namiesto jedného univerzálného agenta využívame sadu sub-agentov (napr. editáciu obsahu a generovanie textu). Každý z týchto agentov disponuje vlastným profilom, čo mu umožňuje pracovať s vyššou mierou spoľahlivosti, keďže sa sústreďí len na úzko vymedzenú časť funkcionality systému.

Kľúčovým prvkom tejto architektúry je proces rozdeľovania úloh, ktorý zabezpečuje *Router Agent*. Ten vystupuje ako riadiaca jednotka zodpovedná za analýzu vstupu a následné priradenie úloh. *Router* v tomto systéme nevykonáva koncovú požiadavku priamo, ale po vyhodnotení hlasového príkazu priradí operáciu najvhodnejšiemu sub-agentovi alebo použije niektorú funkciu editora.

Tento prístup nielen zvyšuje presnosť správneho fungovania textového editora, ale zároveň zabezpečuje modularitu celého riešenia. Vďaka tejto architektúre je možné systém v budúcnosti jednoducho rozširovať o nové funkcie pridaním ďalších špecializovaných modulov bez nutnosti zásadnej rekonštrukcie existujúcej riadiacej logiky.

3.5 Výzvy a limitácie agentických architektúr

Implementácia **agentických systémov** do prostredia textového editora so sebou prináša technické výzvy, ktoré sa pri klasických programoch bežne nevyskytujú. Jednou z najvýznamnejších je **latencia**, teda časová odozva systému. Keďže navrhované riešenie využíva spoluprácu viacerých agentov, každý príkaz musí najprv prejsť analýzou v *Routeri*. Zatiaľ čo jednoduchšie úkony dokáže systém spracovať okamžite, komplexnejšie požiadavky si následne vyžadujú **dodatočné volanie LLM** u konkrétného *sub-agenta*, čo prirodzene zvyšuje celkovú latenciu. Tento proces vyžaduje viacero po sebe idúcich volaní jazykového modelu, čo môže viesť k časovému zdržaniu medzi vyslovením hlasového povelu a jeho reálnym vykonaním v dokumente.

Ďalšou dôležitou výzvou je **predvídateľnosť a spoľahlivosť** celého riešenia. Na rozdiel od pevne naprogramovaných funkcií, ktoré fungujú podľa presne daných pravidiel, sa agentické systémy rozhodujú na základe štatistickej pravdepodobnosti. V kontexte textového editora to znamená, že model môže na ten istý vstup v rôznych situáciách zareagovať mierne odlišne, čo predstavuje riziko nepresnej úpravy, ak agent nesprávne vyhodnotí príkaz. Práve z tohto dôvodu je v našom návrhu dôležitá *dlhodobá pamäť* vo forme mechanizmu **Undo**. Táto funkcia uchováva predchádzajúce verzie textu a slúži ako nevyhnutná **bezpečnostná poistka** v prípade nepredvídateľného správania modelu.

Poslednou bariérou je **nákladovosť** a efektívna **správa kontextu**. Každý agent vyžaduje na svoje správne fungovanie určité množstvo vstupných informácií, ako sú systémové inštrukcie a aktuálny text dokumentu. Pri práci s rozsiahlym textom sa tieto nároky na výpočtový výkon zvyšujú, čo priamo ovplyvňuje stabilitu celého riešenia. [20]

4 Analýza a návrh systému

V tejto kapitole sa zameriame na transformáciu teoretických konceptov do konkrétneho technického návrhu hlasom ovládaného textového editora. Na základe technických špecifikácií systému definujeme **funkčné a nefunkčné požiadavky**, ktoré tvoria základ pre následnú implementáciu celého riešenia.

Kľúčovou časťou kapitoly je **architektonický návrh**, ktorý predstavuje detailný popis štruktúry systému, jeho jednotlivých komponentov a spôsobov, akými medzi sebou komunikujú. Tento návrh integruje multi-agentové riadenie a mechanizmy spracovania prirodzeného jazyka do uceleného celku. Architektúra je navrhnutá tak, aby systém dokázal samostatne vykonávať editačné, navigačné a štrukturálne úlohy priamo v prostredí webovej aplikácie. [2]

4.1 Špecifikácia funkčných požiadaviek

Funkčné požiadavky definujú konkrétne operácie a služby, ktoré systém poskytuje používateľovi. Pre potreby navrhovaného editora boli stanovené nasledujúce kľúčové požiadavky:

FP1 (Snímanie hlasového vstupu): Systém umožňuje používateľovi aktivovať mikrofón priamo v prostredí webového prehliadača a zaznamenávať hlasové príkazy pre ďalšie spracovanie.

FP2 (Interpretácia prirodzeného jazyka): Systém analyzuje neštruktúrované hlasové pokyny a transformuje ich na konkrétne editačné alebo navigačné operácie.

FP3 (Úprava obsahu): Systém vykonáva zmeny v texte (vkládanie, mazanie, prepisovanie) na základe obsahu hlasového príkazu a aktuálneho stavu dokumentu.

FP4 (Generovanie a transformácia textu): Systém dokáže na základe hlasového zadania navrhnúť nový text, rozšíriť existujúci obsah alebo preštylizovať vybranú časť dokumentu pri zachovaní pôvodného kontextu.

FP5 (Navigácia): Systém umožňuje zmenu pozície kurzora v rámci dokumentu pomocou hlasových povelov, ktoré referujú na časti textu (napr. „chod’ na predchádzajúci odsek“).

FP6 (Správa histórie zmien): Systém uchováva predchádzajúce stavy dokumentu a umožňuje používateľovi vrátiť sa k starším verziám (funkcia *Undo*) pomocou hlasového повелu.

FP7 (Vizuálna spätná väzba): Systém v reálnom čase informuje používateľa o stave spracovania hlasového príkazu a o vykonaných zmenách v textovom poli.

4.2 Špecifikácia nefunkčných požiadaviek

Zatiaľ čo funkčné požiadavky hovoria o tom, čo všetko má editor dokázať, nefunkčné požiadavky určujú, ako kvalitne a spoľahlivo má tento proces prebiehať. Pri návrhu systému sme sa zamerali najmä na to, aby bola práca s hlasom plynulá, bezpečná a aby systém zbytočne nezaťažoval výpočtové zdroje. Keďže celá aplikácia beží priamo v prostredí webového prehliadača, bolo nevyhnutné definovať technické limity, ktoré zabezpečia stabilitu editora aj pri spracovaní zložitejších hlasových pokynov. Pre tento účel sme stanovili nasledujúce kritériá:

NP1 (Latencia): Systém musí spracovať hlasový príkaz a začať vykonávať akciu v textovom poli v čase, ktorý negatívne neovplyvňuje používateľskú skúsenosť (ideálne do 3-8 sekúnd v závislosti od rýchlosti siete a úlohy).

NP2 (Dostupnosť a platforma): Editor musí byť plne funkčný v moderných webových prehliadačoch (Chrome, Firefox).

NP3 (Bezpečnosť a súkromie): Systém nesmie ukladať hlasové nahrávky používateľa. Komunikácia medzi klientom a serverom musí byť šifrovaná pomocou protokolu HTTPS.

NP4 (Obnova stavu): Architektúra systému musí minimalizovať dopad chybných interpretácií príkazu. V prípade, že dôjde k nežiaducej úprave textu vplyvom nepresnosti jazykového modelu, systém musí používateľovi poskytnúť okamžitú možnosť návratu k predchádzajúcemu bezchybnému stavu prostredníctvom mechanizmu *Undo*.

NP5 (Správa kontextového okna): Systém musí byť navrhnutý tak, aby efektívne pracoval s limitmi jazykových modelov (vstupné tokeny). Architektúra musí zabezpečiť, aby sa do analýzy posielal len relevantný výsek textu.

NP6 (Správa prístupových kľúčov): Prístupové údaje k službám jazykových modelov sú spravované lokálne v prostredí prehliadača používateľa. Systém tieto kľúče neodosiela na žiadne externé úložiská okrem priamej autorizácie voči oficiálnym API rozhraniam (napr. OpenAI API), čím je zabezpečená kontrola používateľa nad vlastnými prostriedkami.

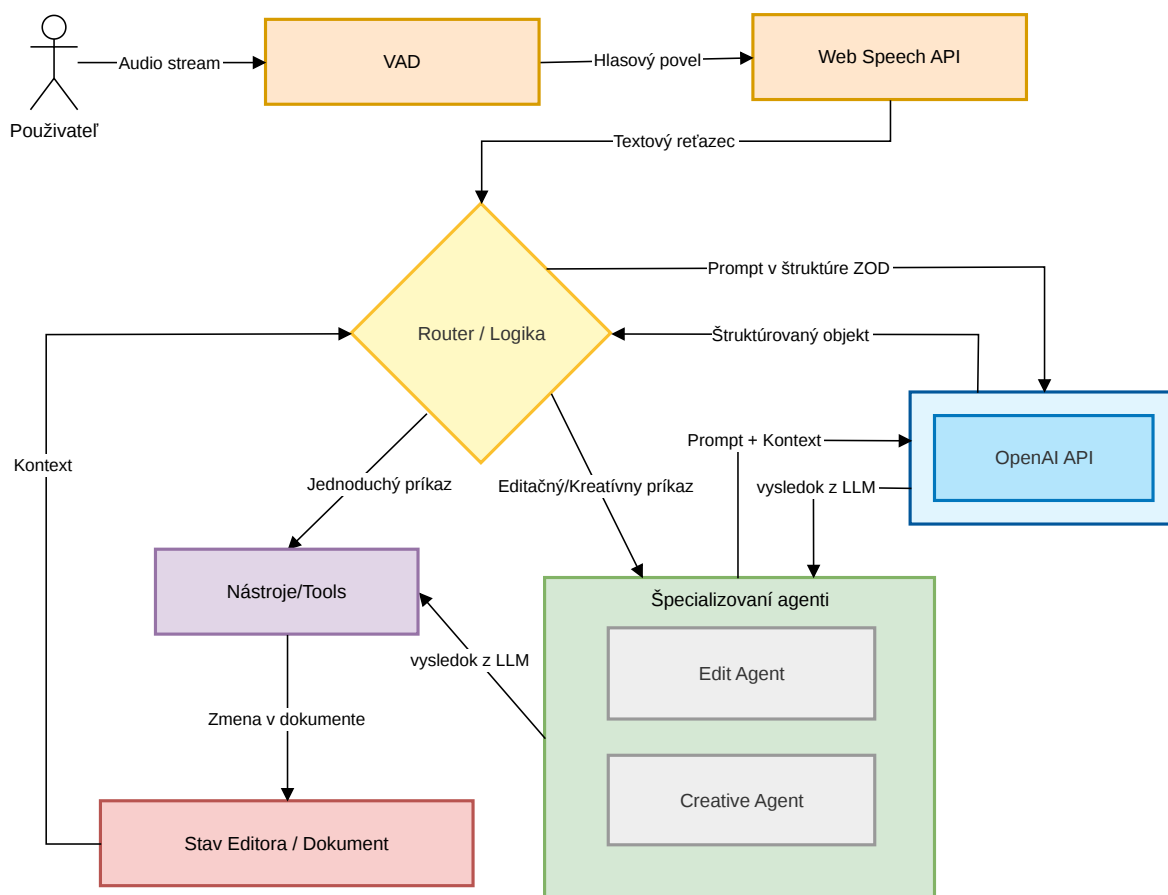
4.3 Architektonický návrh aplikácie

Aplikácia je navrhnutá ako **čisto klientská webová aplikácia (client-side)**, čo znamená, že celá logika spracovania prebieha priamo v prostredí prehliadača používateľa. Tento prístup minimalizuje latenciu, keďže príkazy nemusia prechádzať cez sprostredkovateľský server, a zároveň zvyšuje ochranu súkromia, pretože obsah dokumentu zostáva lokálne uložený. Jedinou formou externej komunikácie je zabezpečené volanie OpenAI API.

Aby editor dokázal inteligentne reagovať na hlasové pokyny, je rozdelený na tri logické časti, pričom každá časť rieši iný problém:

- **Zachytenie a spracovanie hlasu:** Zabezpečuje obsluhu mikrofónu a okamžitú transformáciu reči na text pomocou modelu Whisper. V tejto fáze sa získava textový prepis, ktorý je následne odoslaný *Router Agentovi*.
- **Rozhodovanie:** V tejto vrstve prebieha kľúčová logika spracovania. Systém analyzuje zachytený text a interpretuje zámer používateľa. Na základe tejto interpretácie systém zvolí jednu z dvoch ciest. Ak ide o jednoduchý systémový príkaz (napr. krok späť, zmena veľkosti písma alebo priame dopísanie diktovaného textu), aplikácia ho vykoná okamžite pomocou vopred definovaných funkcií. Ak je príkaz komplexný (napr. požiadavka na preštylizovanie textu), systém deleguje úlohu špecializovanému agentovi.
- **Vykonanie:** Vybraný agent prijme konkrétnu inštrukciu a realizuje zmenu priamo v štruktúre dokumentu. Agent popri používateľskom príkaze berie do úvahy aj **aktuálny kontext**. Ten zahŕňa obsah upravovaného odstavca a polohu kurzora. Vďaka tomu dokáže systém správne interpretovať príkaz a vykonať presnú zmenu v texte.

Výhodou tohto prístupu je popri logickom rozdelení úloh aj vysoká miera modularity, spomínaná v kapitole 3.3.



Obr. 4.1: Schéma toku informácií zachytávajúca cestu od hlasového vstupu cez VAD a model Whisper až po logiku Router Agentu a finálnu úpravu dokumentu.

4.4 Návrh používateľského rozhrania

Pri navrhovaní vizuálnej stránky editora sa kládol dôraz predovšetkým na maximálnu jednoduchosť a prehľadnosť. Keďže hlavným spôsobom interakcie je hlas, cieľom bolo vytvoriť prostredie, ktoré používateľa nerozptyľuje zbytočnými ovládacími prvkami. Celý dizajn je teda postavený na myšlienke, že vizuálne rozhranie má ustúpiť a nechať priestor pre samotnú tvorbu textu. Rozhranie je rozdelené na niekoľko funkčných častí:

- **Konfiguračná časť:** V hornej časti rozhrania sa nachádza pole určené na zadanie API kľúča k službe umelej inteligencie. Toto pole umožňuje prepojenie editora s jazykovým modelom, ktorý zabezpečuje spracovanie a interpretáciu pokynov.
- **Ovládací panel:** Stredná časť obsahuje intuitívne rozhranie pre prácu s pokynmi. Dominantným prvkom je tlačidlo na aktiváciu hlasového snímania, ktoré umožňuje zapnúť alebo vypnúť počúvanie podľa potreby. Okrem hlasového vstupu je k dispozícii aj pole pre ručné zadávanie textových pokynov, čo slúži ako záloha v prípade hlučného prostredia alebo pre presnejšiu kontrolu nad akciami.

- **Editor s nezávislými odseki:** Hlavná časť obrazovky patrí samotnému textovému editoru. Na rozdiel od bežných editorov je text rozdelený do samostatných, nezávislých blokov. Každý odsek má pridelené svoje číslo, čo uľahčuje orientáciu používateľa a umožňuje agentom presnejšie mieriť úpravy. Aktuálny odsek, s ktorým používateľ pracuje alebo na ktorý sa vzťahuje kurzor, je v rozhraní jednoznačne vyznačený modrou farbou. Toto vizuálne odlíšenie pomáha predchádzať chybám pri úpravách a používateľ má vždy okamžitý prehľad o tom, kde sa v štruktúre dokumentu práve nachádza.

Hlavnou výhodou tohto riešenia je odstránenie vizuálneho šumu. Väčšina nástrojov, ktoré sa v bežných editoroch nachádzajú v horných lištách a menu, je v tomto prípade nahradená hlasovými povelmi. Výsledkom je moderné a čisté prostredie, ktoré používateľovi nezavádza a dovoľuje mu plne sa sústrediť na obsah dokumentu.

4.5 Funkčná analýza a spracovanie pokynov

V tejto časti sa definuje, aké konkrétne úlohy dokáže systém vykonávať a ako sú rozdelené medzi jednotlivých agentov. Systém spracováva príkazy rozdelením do kategórií, pričom každá z nich aktivuje iný mechanizmus úpravy:

- **Správa histórie (Undo):** Systém umožňuje vrátiť sa o krok späť v prípade chyby alebo nespokojnosti s výsledkom. Pri každej zmene textu sa aktuálny stav odsekov uloží do histórie, čo používateľovi poskytuje bezpečie pri experimentovaní s textom.
- **Funkcia pre štruktúru:** Funkcia rieši manipuláciu s blokmi textu. Dokáže na základe povelu vymazať konkrétny odsek alebo vložiť nový prázdny odsek pred či za aktuálnu pozíciu.
- **Edit Agent:** Zameriava sa na prepisovanie už existujúceho textu. Na základe pokynu dokáže zmeniť slová a vety, opraviť v ňom chyby alebo ho preštylizovať.
- **Navigačná funkcia:** Zabezpečuje pohyb v dokumente. Interpretuje príkazy na zmenu aktívneho odseku, čím nahrádza manuálne klikanie myšou medzi jednotlivými blokmi textu.
- **Vizuálne nastavenia:** Systém dokáže na základe jednoduchého povelu vykonať okamžitú zmenu veľkosti písma v editore, čo zvyšuje komfort pri práci s textom.
- **Diktovanie:** Slúži na priame dopĺňanie textu. Zachytená reč sa v tomto režime neanalyzuje ako príkaz, ale sa jednoducho pripojí na koniec aktuálneho odseku.

- **Creative Agent:** Tento agent sa využíva na generovanie nového obsahu. Na rozdiel od bežnej úpravy tento modul text nenahrádza, ale vytvára nové nápady alebo pokračovanie textu, ktoré následne pripojí k existujúcemu obsahu.

Všetky tieto funkcie sú riadené hlavným Routerom, ktorý podľa textu príkazu rozhodne, do ktorej kategórie požiadavka patrí. Ak ide o jednoduchý povel, systém ho vykoná okamžite. Ak ide o zložitejšiu prácu s významom, systém zapojí príslušného agenta s umelou inteligenciou.

5 Implementácia a vývoj systému

Kapitola popisuje technický proces realizácie editora, ktorý neprebíhal ako priama implementácia finálneho celku, ale formou postupného vylepšovania a hľadania optimálnych riešení. Pre zostavenie a vývoj aplikácie bol využitý nástroj Vite, ktorý zabezpečuje rýchlu transformáciu zdrojového kódu (TypeScript) do optimalizovanej podoby pre webové prehliadače. Každá podkapitola predstavuje dôležitú vývojovú fázu, ktorá riešila konkrétne technologické obmedzenia predchádzajúcej verzie.

5.1 Prvotný prototyp a deterministické riadenie

V prvej fáze vývoja bolo cieľom vytvoriť funkčný textový editor, ktorý by bolo možné ovládať písanými príkazmi v prirodzenej reči bez nutnosti neustáleho klikania myšou v používateľskom rozhraní. V tejto verzii absentovalo hlasové ovládanie a systém sa spoliehal na striktné definovanú logiku.

5.1.1 Dátová štruktúra a správa stavu

Základom implementácie bolo rozhodnutie upustiť od klasického jednoliateho textového poľa. Namiesto neho je dokument v hlavnom komponente `App.tsx` definovaný ako pole reťazcov (*string array*). Na jeho správu je využitý React hook `useState`, ktorý zabezpečuje reaktivitu a automatické prekresľovanie rozhrania pri každej zmene obsahu:

```
const [paragraphs, setParagraphs] = useState<string[]>([]);
```

Tento prístup bol zvolený namiesto klasického jednocelkového textu z dôvodu, že AI agenti môžu vďaka indexovaniu poľa presne identifikovať konkrétny odsek, ktorý majú upraviť a React vďaka tejto štruktúre aktualizuje len tie časti DOM stromu, v ktorých reálne došlo k zmene.

Okrem textu sú v stave aplikácie spravované aj metaúdaje, ako `currentIndex`, ktorý určuje pozíciu kurzora (index aktuálne spracovávaného odseku).

5.1.2 Vizuálne rozhranie: Komponent `EditorView`

Zobrazenie textového obsahu zabezpečuje komponent `EditorView.tsx`. Jeho primárnou úlohou je iterovať cez pole `paragraphs` a pre každý jeden prvok (odsek) vygenerovať samostatný element `div` s atribútom `contentEditable`. Tento atribút umožňuje používateľovi v prípade potreby manuálne zasahovať do textu pomocou klávesnice a myši. Spätná synchronizácia manuálnych úprav s hlavným stavom aplikácie (`paragraphs`) je realizovaná prostredníctvom udalosti `onBlur`, ktorá pri kliknutí mimo odseku vyvolá funkciu `onManualEdit` a aktualizuje príslušný index v poli.

Pre lepšiu orientáciu a efektívnejšiu navigáciu pomocou prirodzených príkazov je ku každému bloku priradené poradové číslo, ktoré sa zobrazuje vedľa odseku z ľavej strany.

Pre zvýšenie komfortu pri automatizovanej práci s textom je implementované automatické centrovanie (*scroll-into-view*) pomocou hooku `useEffect` a referencie `activeParagraphRef`. Ak systém zmení aktívny odsek (napr. na základe príkazu na navigáciu), rozhranie plynule presunie pohľad tak, aby sa zvýraznený blok nachádzal vždy v strede viditeľnej časti obrazovky.

5.1.3 Prvotná logika rozhodovania

V tejto ranej verzii absentoval komplexný riadiaci agent. Na analýzu písaných vstupov sa využívala detekcia na základe **kľúčových slov**. Ak používateľ zadal slovo „presuň“ alebo „napíš“, systém vyvolal priradeného agenta. Tento prístup bol však značne nespoľahlivý. Akýkoľvek preklep alebo použitie synonyma, ktoré sa nenachádzalo v preddefinovanom slovníku, viedli k zlyhaniu príkazu. Používateľ navyše nemohol v jednom vstupe kombinovať viacero akcií, pretože systém nedokázal logicky rozdeliť komplexnú úlohu na viacero nezávislých operácií.

5.1.4 Implementovanie prvých agentov

Aj keď bolo udeľovanie úloh v tejto fáze primitívne, samotné operácie nad textom už vykonávali špecializovaní agenti (pre navigáciu a editáciu). Pre zabezpečenie jednotnej komunikácie s modelmi LLM bol vytvorený modul `langchainClient.ts`.

Príkladom takéhoto agenta je `editAgent.ts` (5.1). Agent pracuje s tromi základnými vstupmi: kľúčom k API, textom upravovaného odseku a príkazom od používateľa.

Logika spracovania prebieha v troch krokoch:

1. **Definícia identity:** Cez *System Message* sa modelu priradi rola textového editora s jedinou inštrukciou.

2. **Príprava kontextu:** Do správy pre používateľa (*User Message*) sa vložia dáta, pričom pôvodný text odseku je jasne oddelený od príkazu, aby model vedel, čo má modifikovať a čo je inštrukcia.
3. **Exekúcia:** Inicializuje sa model pomocou funkcie `getLCModel` a cez funkciu `callLC` sa odošle požiadavka do LLM, pričom výsledkom je upravený textový reťazec pripravený na vloženie späť do editora. Ukážky funkcií (`getLCModel` 3.1 a `callLC` 3.2) sú v kapitole 3.3.1.

```
1 import { getLCModel, callLC } from "../langchainClient";
2
3 export async function runEditAgent(opts: {
4   apiKey: string;
5   paragraph: string;
6   command: string;
7 }) {
8   const { apiKey, paragraph, command } = opts;
9   const system = 'You are a text editor. Instruction: Modify the paragraph based on users command.';
10
11   const messages = [
12     { role: "system", content: system },
13     {
14       role: "user",
15       content: 'PARAGRAPH TO EDIT:
16
17         ${paragraph}
18
19
20         USER COMMAND:
21         ${command}'
22     }
23   ];
24   const model = getLCModel(apiKey);
25   const output = await callLC(model, messages);
26   return output.trim();
27 }
```

Algoritmus 5.1: Implementácia Edit Agenta

Na podobnom princípe bol implementovaný aj `navigateAgent.ts`. Rozdiel spočíval v tom, že namiesto textového kontextu odseku pracoval s číselnými metadátami – aktuálnym indexom a celkovým počtom odsekov. Jeho systémová inštrukcia striktne obmedzovala výstup modelu na jediné číslo, ktoré reprezentovalo cieľový odsek: `You are a navigation agent. Respond with ONLY a single number from 1 to ${total}.`

Prompty v pôvodnom, minimalistickom formáte vykazovali pri testovaní menšie chyby. Model mal tendenciu do výstupu pridávať zdvorilostné frázy (napr. „Nech sa páči, tu je upravený text.“), uvádzať text v úvodzovkách alebo v prípade navigácie odpovedať celou vetou namiesto samotného čísla.

Z tohto dôvodu boli systémové inštrukcie iteratívne upravené do finálnej podoby, ktorá striktne definuje formu:

- **Edit Agent:** Inštrukcia bola rozšírená o explicitné zákazy metadát:
"Return ONLY the modified text. No explanations, no quotation marks, no meta-comments."
- **Navigate Agent:** Pôvodné číselné obmedzenie sa ukázalo ako nedostatočné pre relatívny pohyb. Prompt bol prepracovaný tak, aby agent dokázal pracovať s abstraktnými pojmami (*next*, *prev*) aj priamym určením indexu:
"Use: 'next', 'prev', 'first', 'last'. If user names a specific number (e.g. 'go to paragraph 5'), return that number as a string. Current index is $\{currentIndex + 1\}$, total paragraphs: $\{total\}$."

Model sa vďaka presnejšiemu vymedzeniu mantinelov stal spoľahlivejším a presnejším.

Hoci tento prístup v počiatočnej fáze fungoval, s narastajúcim počtom funkcií editora sa začali prejavovať jeho závažné nedostatky. Pri rozširovaní možností úpravy a štruktúry textu sa metóda založená na kľúčových slovách stala nepresnou, pretože pri komplexnejších úkonoch dochádzalo k sémantickým kolíziám. Rôzni agenti by pri určitých poveloch vyžadovali rovnaké kľúčové slová. Systém navyše nedokázal pochopiť komplexný príkaz (napr. „oprav chyby a potom to posuň nižšie“) bez toho, aby ho používateľ musel manuálne rozdeliť na jednotlivé kroky.

5.2 Integrácia riadiaceho agenta

Na elimináciu nepredvídateľnosti odpovedí pri implementácii kľúčového **Router Agent**a bola využitá technológia *Structured Output*. Hlavným pilierom je knižnica *zod*, pomocou ktorej sme definovali `taskScheme`.

```

1  const taskScheme = z.object({
2    tasks: z.array(
3      z.object({
4        category: z.enum(["edit", "navigate"])
5        .describe("- edit: Modifying existing text, - navigate: Moving between paragraphs"),
6        cleanCommand: z.string()
7        .describe("Clear, grammatically correct version of the instruction"),
8      })
9    )
10 });

```

Algoritmus 5.2: Definícia schémy `taskScheme`

5.2.1 Vynútenie štruktúry a exekúcia

Metóda `.withStructuredOutput(taskScheme)` zabezpečuje prepojenie modelu s touto schémou. Funguje ako inteligentný filter, ktorý zaručuje, že *LLM* nevráti voľný text, ale vráti dáta striktne podľa stanovených pravidiel.

Pri dopytovaní už nevyužívame pomocnú funkciu `callLC`, pretože nepotrebujeme surový textový reťazec. Namiesto toho komunikujeme s modelom priamo cez metódu `.invoke()`. Výsledkom je už validovaný objekt, ktorý riadiaca logika aplikácie môže okamžite použiť.

```
1 const model = getLCModel(apiKey);
2 const structuredLlm = model.withStructuredOutput(taskSchema);
3
4 const system = `You are an Intelligent Task Planner for a text editor.
5 Your goal is to decompose the users voice command into a sequence of atomic tasks.`;
6 const decision = await structuredLlm.invoke([
7   { role: "system", content: systemPrompt },
8   { role: "user", content: Command: `${command}` }
9 ]);
```

Algoritmus 5.3: Volanie štruktúrovaného modelu v Router Agentovi

Ďalším prínosom tejto fázy je schopnosť modelu vrátiť pole úloh. Ak používateľ zadá komplexný príkaz, napríklad: „*Oprav chyby v tomto odseku a potom skoč na ďalší*“ model vráti pole s dvoma objektmi:

1. `category: "edit", ...` – delegované na `Edit Agent`.
2. `category: "navigate", ...` – delegované na `Navigate Agent`.

Systém následne v cykle prechádza toto pole a postupne vykonáva akcie. Tento prístup vyriešil problém so synonymami, keďže model sa pri klasifikácii riadi významom vety, nie prítomnosťou konkrétneho kľúčového slova. Týmto sa rozhranie stalo skutočne prirodzeným a odolným voči variabilite ľudskej reči.

5.3 Pridanie ďalších funkcií editora a optimalizácia

S funkčným riadiacim agentom sa vývoj sústredil na rozšírenie možností editora a zvýšenie jeho efektivity. Cieľom tejto fázy bolo umožniť systému nielen text opravovať, ale ho aj aktívne vytvárať a spravovať jeho štruktúru bez zbytočnej latencie.

5.3.1 Creative Agent: Generovanie obsahu a kreativita

Pre úlohy zamerané na tvorbu nového textu, ako je pokračovanie v rozpísanej myšlienke alebo písanie úplne nového textu, bol vytvorený špecializovaný `CreativeAgent.ts`. Na rozdiel od *Edit Agent*a, ktorý je striktne deterministický, tento agent využíva model inicializovaný s vyššou mierou náhodnosti (`temperature: 0.8`).

Agent v rámci svojho systémového promptu dostáva inštrukciu, aby analyzoval kontext aktuálneho odseku a plynule naň nadviazal, pričom musí zachovať štýl a tón autora. Ukážka implementácie tohto agenta bola v kapitole 3.3.3 algoritmus 3.3.

5.3.2 Rozširovanie funkcií editora a znižovanie latencie

Počas testovania sa ukázalo, že pôvodná separácia na špecializovaných agentov pre každú drobnú operáciu je neefektívna. Každé dodatočné volanie jazykového modelu prináša latenciu v rádoch sekúnd, čo negatívne ovplyvňuje používateľskú skúsenosť.

Zatiaľ čo **Edit Agent** (zameraný na gramatické a sémantické úpravy) a **Creative-Agent** (určený na generovanie textu) majú v systéme svoje opodstatnenie kvôli potrebe špecifického nastavenia modelu (teplota, promptovanie), ich hlavným prínosom je najmä úzka špecializácia. Tá modelu umožňuje sústrediť sa na jasne vymedzený rozsah inštrukcií, čím sa výrazne zvyšuje jeho presnosť a spoľahlivosť. Naopak, úlohy spojené s jednoduchou navigáciou a technickou správou editora sú pre moderné LLM triviálne, čo umožnilo ich integráciu do riadiaceho modulu.

Z tohto dôvodu bol **Navigate Agent** úplne odstránený. Jeho logika bola integrovaná priamo do **Router Agent**a, ktorý v rámci jedného kroku dokáže pochopiť príkaz a zároveň z neho extrahovať potrebné technické parametre. Pre plnohodnotné ovládanie editora bez klávesnice sme do schémy `taskScheme` integrovali aj ovládanie vizuálnych vlastností a štruktúry dokumentu.

- **Undo (Krok späť):** Pre zabezpečenie bezpečnosti pri experimentovaní s AI bol implementovaný stav `history`. Pri každej deštruktívnej zmene (editácia, zmazanie) sa aktuálne pole `paragraphs` uloží do zásobníka. Ak **Router Agent** rozpozná kategóriu `undo`, systém jednoducho vráti predchádzajúci stav zo zásobníka.
- **Veľkosť písma:** Príkazy ako „zväčši písmo o 5“ sú spracované okamžite. Model totiž nemusí zložito vymýšľať novú vetu, stačí mu z príkazu vytiahnuť len dve dôležité informácie: čo má urobiť (zväčšiť) a o koľko (o 5 bodov). Tieto údaje sa rovno pošlú do vizuálnej časti editora, ktorá upraví veľkosť písma.
- **Štruktúrne zmeny:** Kategória `structure` umožňuje manipuláciu s poľom odsekov (pridávanie prázdnych blokov pred/za aktívny odsek alebo ich mazanie).
- **Diktovanie (Dictate):** Táto kategória slúži na priame vkladanie surového prepisu reči do aktuálneho odseku, pričom model zabezpečí odstránenie výplňových slov, ktoré sa v reči prirodzene vyskytujú.

```
1  const taskScheme = z.object({
2    tasks: z.array(
3      z.object({
4        category: z.enum([
5          "edit", "creative", "navigate", "structure", "undo", "font", "dictate"
6        ]).describe('
7          - edit: Modifying existing text (grammar, tone, translation).
8          - creative: New content, poems, metaphors, or story continuation.
9          - navigate: Moving between paragraphs.
```

```

10         - structure: Creating or deleting entire paragraphs.
11         - undo: Reverting the last action.
12         - font: Changing font size.
13         - dictate: Adding raw dictated text to current paragraph.
14     '),
15     cleanCommand: z.string().describe("Clear, grammatically correct version of the user's
instruction in " + langName + ". None if not creative, or edit."),
16
17     creativeMode: z.enum(["append", "replace", "none"])
18     .describe("Only for 'creative' category: 'append': User wants to continue the story.
'replace': User wants to rewrite."),
19
20     structureAction: z.enum(["add_after", "add_before", "delete", "none"])
21     .describe("Only for 'structure' category. What to do with the paragraph. 'none' means no
structural change."),
22
23     fontAction: z.enum(["increase", "decrease", "reset", "none"])
24     .describe("Only for 'font' categories. How to adjust the font size. 'none' means no
change."),
25
26     fontValue: z.number()
27     .describe("The numeric value by which to change the font (e.g., 3, 5, 10). If not specified,
default to 2. 'none' means no change."),
28
29     navTarget: z.string()
30     .describe("Target for 'navigate' category.
31         Use: 'next', 'prev', 'first', 'last'. If user names a specific number, return that number
as a string. Current index is ${currentIndex + 1}, total paragraphs: ${total}."),
32
33     dictateContent: z.string()
34     .describe("Only for 'dictate' category.
35         Extract the ACTUAL text the user wants to write. STRICTLY REMOVE all introductory phrases
like 'write', 'input'... ")
36     }));

```

Algoritmus 5.4: Kompletná definícia taskScheme so všetkými funkciami

5.4 Implementácia hlasového rozhrania a spracovanie audia

Prechod na hlasové ovládanie vyžadoval integráciu technológií na spracovanie signálu v reálnom čase. Hlavnou výzvou bolo nájsť rovnováhu medzi rýchlosťou odozvy a presnosťou prepisu hlasu na text (*STT – Speech-to-Text*).

5.4.1 Manuálne nahrávanie (Whisper API)

Prvotným prístupom k implementácii hlasového ovládania bolo vytvorenie tradičného rozhrania založeného na manuálnom spúšťaní nahrávania. V tejto fáze sme využili natívne prehliadačové rozhranie `MediaRecorder`, ktoré umožňovalo zachytávať audio stream z mikrofónu používateľa. Technicky bol proces spracovaný tak, že prichádza-

júce zvukové dáta sa ukladali do dočasného poľa fragmentov. Po manuálnom ukončení nahrávania stlačením tlačidla sa tieto fragmenty spojili do jedného objektu typu `Blob` s kódovaním `audio/webm`, z ktorého sa následne vytvoril súbor pripravený na odoslanie.

Kľúčovým prvkom pre transformáciu zvuku na text sa stal model `whisper-1` od spoločnosti OpenAI. Implementovali sme funkciu `sendToWhisper`, ktorá pomocou objektu `FormData` zapuzdrla audio súbor spolu s konfiguračnými parametrami, ako je identifikátor modelu.

```
1 async function sendToWhisper(file: File): Promise {
2   const formData = new FormData();
3   formData.append("file", file);
4   formData.append("model", "whisper-1");
5
6   const res = await fetch("https://api.openai.com/v1/audio/transcriptions", {
7     method: "POST",
8     headers: { Authorization: Bearer ${apiKey} },
9     body: formData
10  });
11
12  const data = await res.json();
13  return data.text || "";
14 }
```

Algoritmus 5.5: Odosielanie audio dát na API Whisper

Hoci tento model vykazoval vysokú presnosť pri prepise, samotný proces manuálneho nahrávania sa v praxi ukázal ako problematický. Používateľ bol nútený neustále sledovať stav nahrávania a manuálne ho pri každom príkaze obsluhovať. Tento mechanizmus bol preto vyhodnotený ako neprirodzený pre plynulú prácu s textom, no definoval základnú komunikačnú vrstvu s OpenAI API, ktorú sme neskôr úspešne adaptovali aj pri implementácii automatizovaného rozpoznávania aktivity (*VAD*).

5.4.2 Streaming a inteligentné ukončovanie

Cieľom druhej fázy vývoja bolo odstrániť potrebu manuálneho ovládania nahrávania a dosiahnuť pocit plynulejšej, takmer okamžitej interakcie. Na tento účel sme nasadili technológiu od spoločnosti **Deepgram** [3] s využitím modelu *Nova-2*. Hlavná inovácia spočívala v prechode z jednorazového odosielania súborov na kontinuálne streamovanie audia prostredníctvom protokolu *WebSocket*.

Tento prístup umožnil aplikácii odosielať malé fragmenty zvukových dát v reálnom čase (každých 100ms). Najväčším prínosom bolo využitie parametra `endpointing`, ktorý sme nastavili na hodnotu 500ms. Vďaka nemu dokázal vzdialený server sám identifikovať prirodzené ticho po skončení používateľovho príkazu a automaticky vrátiť finálny prepis. Tým sa eliminovala nutnosť manuálneho zastavovania nahrávania a systém začal pôsobiť inteligentnejšie.

Napriek extrémnej rýchlosti a nízkej latencii sa však v tejto fáze objavili kvalita-
tívne nedostatky. Model *Nova-2* vykazoval pri slovenskom, ale aj pri anglickom jazyku
vyššiu mieru chybovosti v porovnaní s modelom Whisper. Často dochádzalo k nespráv-
nej interpretácii termínov alebo skomoleniu slov pri horších podmienkach, čo následne
spôsobovalo nesprávnu kategorizáciu úloh v *Router Agentovi*. Práve táto skúsenosť
nás viedla k hľadaniu ďalšieho riešenia, ktoré by kombinovalo používateľský komfort
Deepgramu s vysokou presnosťou modelu Whisper.

5.4.3 Integrácia VAD a Whisper API

Finálne riešenie hlasového rozhrania predstavuje kombináciu presnosti modelu Whisper
a automatizácie pomocou Silero VAD. Tento prístup definitívne rieši nedostatky pred-
chádzajúcich verzií, predovšetkým nutnosť manuálnej obsluhy nahrávania. Kľúčovým
technologickým pilierom sa stala integrácia knižnice `@ricky0123/vad-react`.

Pre dosiahnutie prirodzenej interakcie sme implementovali konfigurovateľné časové
okno ticha prostredníctvom parametra **`redemptionMs` nastaveného na 1000 ms**.
Akonáhle systém deteguje ticho trvajúce dlhšie ako jednu sekundu, segment reči sa
považuje za ukončený. V tomto momente dochádza k technickému spracovaniu, kedy sa
surové dáta typu `Float32Array` automaticky konvertujú do štandardizovaného formátu
WAV a prostredníctvom funkcie `processAudio(blob)` sa posielajú na ďalšie spracovanie.

Veľkou výzvou pri hlasovom ovládaní bolo, ako systém spracuje príkazy, ktoré idú
rýchlo po sebe. Ak by používateľ nadiktoval dva príkazy tesne po sebe, hrozilo by, že
nastane kolízia (tzv. *race condition*) a editor by ich vykonal v zlom poradí alebo by
jednu z nich úplne vynechal.

Tento problém sa vyriešil vytvorením virtuálneho poradovníka (*fronty*). Každý za-
chytený hlasový príkaz sa zaradi do radu a počká, kým sa dokončí spracovanie predchá-
dzajúceho príkazu. Vďaka tomu má systém istotu, že sa príkazy na server odosielaajú a
v editore vykonávajú presne v takom poradí, v akom boli používateľom vyslovené.

```
1  async function initVAD() {
2      const { MicVAD } = window.vad;
3
4      const myVad = await MicVAD.new({
5          startOnLoad: false,
6          redemptionMs: 1000,
7          baseAssetPath: "https://cdn.jsdelivr.net/npm/@ricky0123/vad-lib@0.0.19/dist/",
8          onSpeechEnd: (audio: Float32Array) => {
9
10             const wavBuffer = window.vad.utils.encodeWAV(audio);
11             const blob = new Blob([wavBuffer], { type: "audio/wav" });
12
13             processAudio(blob);
14         },
15     });
16     vadRef.current = myVad;
17 }
```

Po úspešnej detekcii konca reči a vygenerovaní audio súboru nasleduje volanie funkcie `processAudio(blob)`. Spracovanie prebieha vnútorne cez model Whisper, pričom sa využíva takmer rovnaký princíp komunikácie, aký bol opísaný v časti pri manuálnom nahrávaní (5.5).

5.5 Zhrnutie implementačnej fázy

Navrhnutá architektúra sa v praxi ukázala ako opodstatnená a efektívna pre potreby inteligentného editora. Nasadenie technológie *VAD* úspešne eliminovalo potrebu manuálneho ovládania nahrávania, zatiaľ čo integrácia modelu *Whisper* zabezpečila vysokú mieru presnosti pri spracovaní prirodzenej reči. Dôležitým prvkom sú vizuálne indikátory stavu, ktoré používateľovi poskytujú okamžitú spätnú väzbu o prebiehajúcej fáze rozpoznávania reči alebo generovania odpovede agentom.

Celý proces spracovania hlasových pokynov používateľa tvorí uzavretý, logicky nadväzujúci cyklus, detailne vizualizovaný v diagrame architektúry (Obrázok 4.3 v kapitole 4.3). Reťazec operácií začína pri lokálnom modeli *VAD*, pokračuje prepisom cez model *Whisper* a končí pri *Router Agentovi*, ktorý analyzuje príkaz. Ten následne deleguje exekúciu na špecializovaného agenta, ktorý vykoná finálnu úpravu poľa odsekov.

5.6 Používateľské rozhranie finálnej aplikácie

V tejto časti uvádzame vizuálne spracovanie výsledného systému. Cieľom rozhrania bolo poskytnúť maximálnu plochu pre prácu s textom, pričom ovládacie prvky a indikátory stavu sú diskretné umiestnené tak, aby nenarušovali používateľský zážitok.



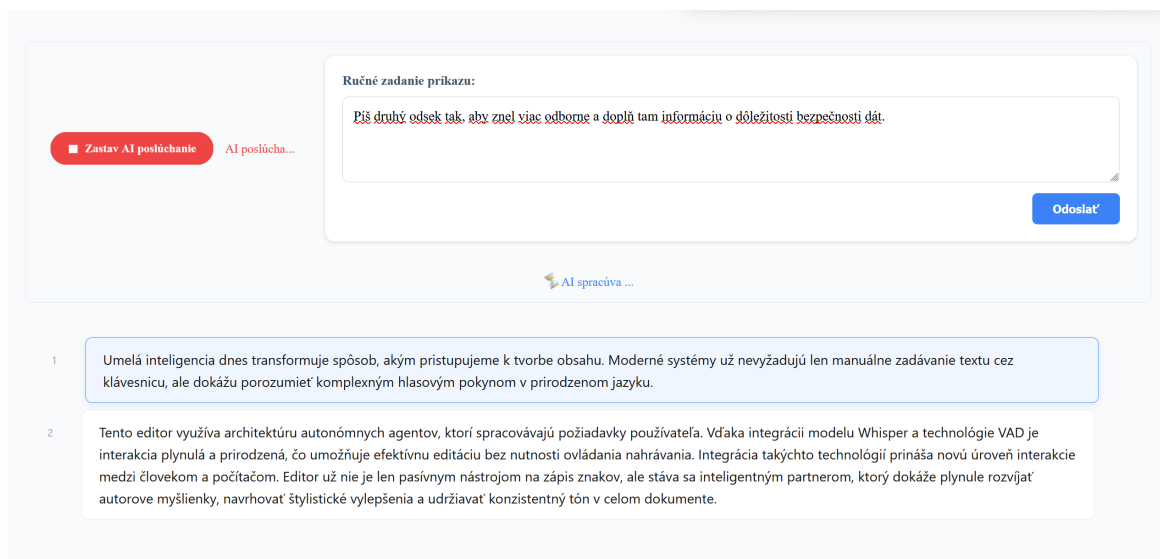
Obr. 5.1: Hlavné rozhranie editora s rozdelením na konfiguračnú časť (API kľúč), textovú plochu s číslovanými odsekmi a ovládacie prvky pre hlasový aj textový vstup.

Na obrázku 5.1 môžeme vidieť okno aplikácie, ktorému dominuje textové pole rozdelené na očíslované bloky. V hornej časti rozhrania sa nachádza konfiguračné pole pre vloženie API kľúča, ktorý je nevyhnutný na autentifikáciu voči službám jazykových modelov.

Ďalšia časť rozhrania je rozdelená na dve funkčné zóny pre interakciu s používateľom. V ľavej časti sa nachádza ovládacie prvky pre aktiváciu mikrofónu, ktorý po zapnutí spúšťa mechanizmus VAD a umožňuje plynulé hlasové zadávanie príkazov. Pravá časť obsahuje textové vstupné pole určené na manuálne zadávanie textových pokynov pomocou klávesnice, čo slúži ako alternatíva v hlučnom prostredí alebo pre precíznu editáciu.

V ľavej časti samotného editora sa nachádzajú identifikátory odsekov, ktoré uľahčujú navigáciu pomocou príkazov. V pravom hornom rohu sú umiestnené ovládacie prvky pre výber jazyka (slovensky, anglicky). Aktuálne vybraný odsek je vizuálne odlíšený modrým zvýraznením, čo poskytuje okamžitú spätnú väzbu o tom, na ktorú časť textu sa budú vzťahovať nasledujúce príkazy.

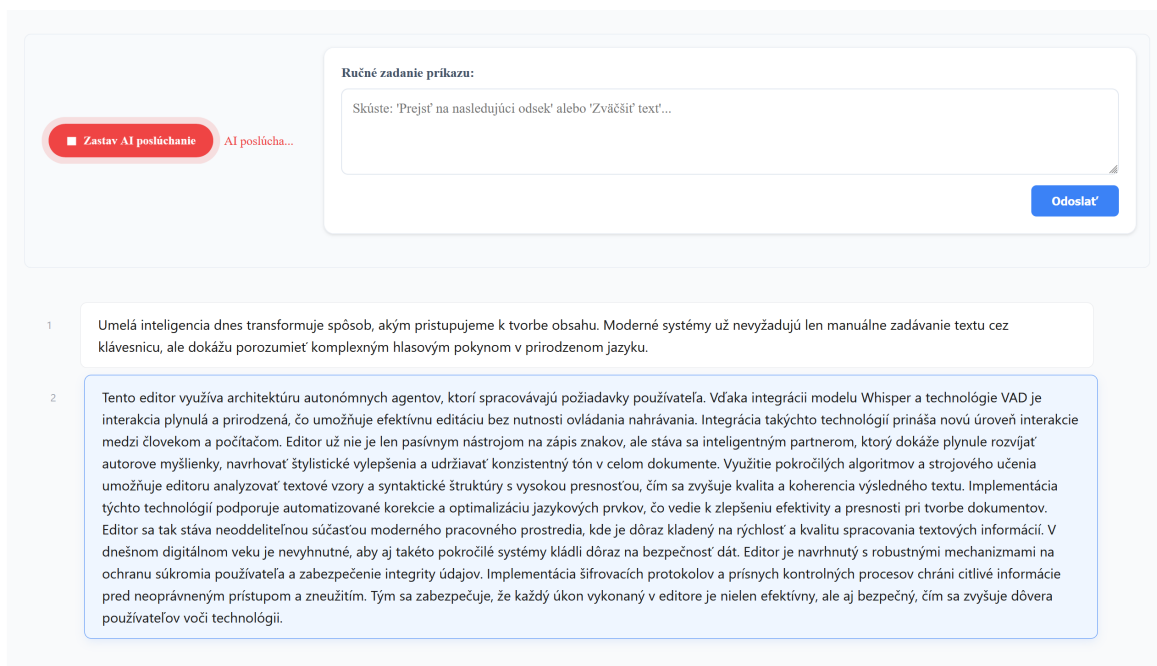
Dôležitým aspektom používateľskej skúsenosti je vizuálna odozva na hlasovú aktivitu a spracovanie úloh. Na obrázku 5.2 je zachytený stav aplikácie počas interakcie s používateľom.



Obr. 5.2: Ukážka vizuálnej odozvy systému. Textové pole zobrazuje prepísaný príkaz z modelu Whisper a stavový riadok indikuje fázu spracovania textu agentmi.

Hneď po manuálnej aktivácii mikrofónu systém informuje používateľa nápisom „*AI počúva...*“, čím potvrdzuje pripravenosť na prijímanie hlasových pokynov. Po detekcii ukončenia reči algoritmus Silero VAD aktivuje prepis audia. V tomto momente sa surový výstup z modelu Whisper okamžite zobrazí v textovom vstupnom poli, čo používateľovi poskytuje vizuálnu kontrolu nad tým, ako bol jeho príkaz interpretovaný.

Následne, počas exekúcie príkazu agentmi, sa vytvorí nový stavový indikátor „*AI spracúva...*“. Po úspešnom vyhodnotení *Router Agentom* a vykonaní zmien príslušným pod-agentom sa upravený text premietne priamo do editora a stavové pole sa vyčistí pre ďalšiu interakciu.



Obr. 5.3: Stav dokumentu po exekúcii príkazu, kde je vidieť úspešná modifikácia konkrétného odseku pri úplnom zachovaní štruktúry textu.

Porovnaním predchádzajúcich stavov s obrázkom 5.3 je zrejmé, že systém úspešne modifikoval cieľový odsek bez narušenia okolitej štruktúry dokumentu. Úspešné vykonanie príkazu v reálnom čase ukazuje, že systém je správne prepojený a dokáže plynule reagovať na požiadavky používateľa.

6 Testovanie a zhodnotenie výsledkov

Cieľom tejto kapitoly je overenie funkčnosti navrhnutej architektúry a zhodnotenie jej pripravenosti na reálne nasadenie. Testovanie sa zameriava nielen na overenie správnosti vykonávania hlasových príkazov v rôznych scenároch, ale aj na analýzu kľúčových výkonnostných parametrov, akými sú latencia spracovania a presnosť interpretácie úloh riadiacim agentom. Kapitola zároveň poskytuje kritický pohľad na súčasné limity systému a identifikuje oblasti pre jeho ďalšie potenciálne vylepšovanie.

6.1 Verifikácia funkčnosti na vybraných scenároch

V tejto časti overujeme funkčnosť systému pomocou testovania rôznych variácií používateľských príkazov. Scenáre boli navrhnuté tak, aby simulovali prirodzené používanie editora, pričom sme sledovali úspešnosť celého reťazca: od akustického spracovania (*Whisper*) cez sémantickú klasifikáciu (*Router Agent*) až po samotné vykonanie akcie špecializovaným agentom.

Testovanie prebiehalo v prostredí s bežným hlukom v pozadí, s cieľom preveriť robustnosť systému voči fonetickým nepresnostiam a variabilite slovenskej reči. Výsledky sú rozdelené do nasledujúcich funkčných kategórií.

6.1.1 Testovanie navigácie

V tejto časti sme podrobili navigačný subsystém testovaniu v troch kľúčových kategóriách, ktoré pokrývajú najčastejšie spôsoby interakcie s textovým dokumentom.

Kategória	Počet testov	Správne vykonal	Úspešnosť (%)
Priama (čísla)	5	5	100 %
Relatívna (posun)	6	5	83,3 %
Extrémy (štart/cieľ)	5	4	80 %
Celkovo	16	14	87,5 %

Tabuľka 6.1: Štatistické vyhodnotenie navigačných scenárov

Analýza neúspešných prípadov Hoci bola celková úspešnosť vysoká, v dvoch prípadoch systém zlyhal. Pri príkaze „**Chod’ hore**“ (kategória Relatívna) bol identifikovaný primárny problém vo fáze *Speech-to-Text*. Model Whisper túto krátku dvojicu slov opakovane skomolil (napr. na „*KUČ HORE!*“) natoľko, že *Router Agent* nebol schopný rozpoznať zámer a príkaz vyhodnotil ako diktovanie textu. Pri ručnom zadaní identického textu do systému bol príkaz interpretovaný bezchybne, čo potvrdzuje správnosť logiky agenta, no zároveň odhaľuje limity akustického spracovania veľmi krátkych fráz.

Druhý neúspešný pokus nastal pri príkaze „**Daj ma úplne hore**“ (kategória Extrémy). V tomto prípade model Whisper vykonal prepis relatívne presne („*Daj ma úplne horé*“), avšak *Router Agent* priradil inštrukciu k nesprávnej kategórii **font**. Došlo k zámene významu slova „hore“ so zvyšovaním hodnoty, čo viedlo k zväčšeniu písma namiesto presunu na začiatok dokumentu. Táto chyba poukazuje na viacvýznamovosť niektorých slovenských výrazov v kontexte riadenia rozhrania.

6.1.2 Testovanie správy dokumentu a vizuálnych vlastností

V druhej fáze verifikácie sme sa zamerali na príkazy ovplyvňujúce štruktúru dokumentu (**structure**), zmeny veľkosti písma a schopnosť systému vrátiť zmeny do pôvodného stavu (**undo**).

Kategória	Počet testov	Správne vykonal	Úspešnosť (%)
Štruktúra (pridávanie)	5	5	100 %
Štruktúra (mazanie)	5	3	60 %
Veľkosti písma	10	10	100 %
História (Undo)	6	5	83,3 %
Celkovo	26	23	88,4 %

Tabuľka 6.2: Štatistické vyhodnotenie správy dokumentu a formátovania

Analýza neúspešných prípadov Podobne ako pri navigácii, aj pri mazaní odsekov bol hlavným zdrojom zlyhaní model *Speech-to-Text*. Príkaz „**Vymaž tento odsek**“ bol modelom Whisper opakovane prepísaný ako „*Vy máš tento odsek*“. Kvôli tejto fonetickej zámene nedokázal *Router Agent* identifikovať kľúčové sloveso pre akciu **delete**. Rovnaká situácia nastala pri príkaze „**Zmaž to**“, kde Whisper nedokázal správne identifikovať slovo „zmaž“ v krátkom kontexte. Po ručnom zadaní príkazu však riadiaca logika v oboch prípadoch okamžite priradila správnu kategóriu **structureAction**: "delete".

6.1.3 Testovanie inteligentného spracovania a úpravy obsahu

Zamerali sme sa na komplexné funkcie systému, ktoré využívajú plný potenciál jazykového modelu. Testovanie zahŕňalo kreatívne rozširovanie textu (**creative**), priame vkladanie obsahu (**dictate**) a hĺbkovú sémantickú úpravu textových blokov (**edit**). Tieto kategórie predstavujú jadro inteligentného editora a vyžadujú najvyššiu mieru presnosti pri analýze kontextu.

Kategória	Počet testov	Správne vykonal	Úspešnosť (%)
Creative Agent (creative)	8	8	100 %
Diktovanie (dictate)	8	5	62,5 %
Úprava textu (edit)	15	11	73,3 %
Celkovo	31	24	77,4 %

Tabuľka 6.3: Štatistické vyhodnotenie inteligentných funkcií editora

Analýza neúspešných prípadov Kategória kreatívneho agenta dosiahla stopercentnú úspešnosť, pričom systém preukázal schopnosť nielen plynule nadviazať na existujúci text, ale aj úspešne generovať nový obsah na základe príkazu používateľa.

Pri kategórii **dictate** sa opäť potvrdilo, že najslabším článkom reťazca je fonetická citlivosť modelu *Whisper*. Príkazy začínajúce slovami ako „Kúpiť“ alebo „Schôdza“ boli často skomolené (napr. „Púpiť“, „Hôza“), čo viedlo k zápisu foneticky podobných, no vecne nesprávnych výrazov do dokumentu. Naopak, pozitívnym zistením bola robustnosť voči výplňovým slovám (napr. „Ehm, no...“), ktoré model dokázal v kontexte diktovania úspešne odfiltrovať.

6.1.4 Testovanie komplexných a sekvenčných príkazov

V poslednej fáze sme preverili schopnosť systému spracovávať viaczložkové inštrukcie, ktoré vyžadujú viacero úkonov súčasne. Testovanie bolo zamerané na príkazy kombinujúce navigáciu s editáciou, paralelnú úpravu vizuálnych a textových vlastností, a vykonávanie opakovaných operácií v rámci jedného hlasového vstupu. Táto úroveň verifikácie simuluje reálne používanie systému, kde používateľ prirodzene reťazí požiadavky.

Kategória	Počet testov	Správne vykonal	Úspešnosť
Zmena textu na inom odseku	4	4	100 %
Štruktúra + Obsah	5	3	60 %
Sekvenčné a opakované akcie	5	4	80 %
Vizuál + Text	5	4	80%
Komplexné reťazce (> 2 akcie)	5	4	80 %
Celkovo	24	19	79,1%

Tabuľka 6.4: Štatistické vyhodnotenie komplexných a sekvenčných operácií

Analýza neúspešných prípadov Pri testovaní viaczložkových príkazov sa potvrdilo, že *Router Agent* dokáže efektívne dekomponovať aj zložité vety na sekvenciu úloh, pokiaľ je vstupný text správne prepísaný. Hlavným zdrojom zlyhaní bol opäť model *Whisper*. Problémy mal so slovami, ako sú sprav, vymaž a používaním cudzích slov, ako je napríklad "undo".

Zaujímavým zistením je, že pri dlhších vetách bola úspešnosť prepisu vyššia než pri krátkych úsečných poveloch. Väčší kontext vety pomohol modelu lepšie predpovedať správne slová, vďaka čomu systém bezchybne vykonal aj trojzložkové operácie.

6.1.5 Zhrnutie verifikačnej fázy

Celkovo bolo v rámci všetkých testovacích blokov vykonaných 97 testovacích scenárov s celkovou priemernou úspešnosťou 82,5 %. Výsledky jasne naznačujú, že navrhnutá architektúra s využitím agentov a LLM routera je vysoko odolná voči variáciám v prirodzenej reči. Významným zistením testovania je schopnosť *Router Agent*a správne identifikovať zámer používateľa aj v prípadoch, kedy *STT* model neposkytol úplne presný prepis. Vďaka kontextuálnej inteligencii jazykového modelu dokázal systém často vykonať správnu akciu aj pri nepresnom prepise, pokiaľ zostal zachovaný celkový význam príkazu. Hlavným priestorom pre zlepšenie ale stále zostáva fáza prepisu reči pri krátkych slovenských výrazoch.

Doplnkové testovanie s identickou sadou príkazov v anglickom jazyku vykázalo stabilitu na úrovni 95,8 %. Tento rozdiel potvrdzuje predpoklad, že celková úspešnosť slovenského rozhrania je limitovaná najmä súčasným stavom technológií pre rozpoznávanie reči. Hoci pri samotnej práci agentov s textom stále existuje mierna odchýlka medzi jazykmi, tá je v porovnaní s chybovosťou pri diktovaní minimálna. Anglický prepis vykazoval výrazne vyššiu presnosť najmä pri krátkych poveloch, ktoré v slovenčine predstavovali hlavný zdroj zlyhaní.

6.2 Analýza latencie a odozvy systému

V tejto časti analyzujeme časovú náročnosť spracovania hlasových príkazov od momentu ukončenia nahrávania až po finálne vykonanie úlohy v editore. Celková latencia je v našej architektúre súčtom troch hlavných fáz: prepisu reči (*STT*), klasifikácie úloh (*Router*) a samotnej exekúcie požiadavky (*Task*). Pre dosiahnutie relevantných výsledkov bola každá kategória zastúpená viacerými scenármi, pričom každá jednotlivá úloha bola testovaná v troch opakovaniach a následne spriemerovaná.

Typ príkazu	STT (ms)	Router (ms)	Task (ms)	Celkom (ms)
Jednoduché (Navigácia, undo)	1 950	1 350	0*	3 300
Zmena veľkosti písma	2 050	1 700	0*	3 750
Diktovanie	2 650	1 800	0*	4 450
Editácia	2 050	2 300	1 750	6 100
Kreatívne (Generovanie)	2 100	1 450	5 150	8 700
Komplexné (Viac úloh)	2 700	2 100	1 900	6 700

Tabuľka 6.5: Namerané hodnoty priemernej latencie podľa kategórií úloh.

(*Pozn.: Nulová hodnota exekúcie značí okamžité vykonanie lokálnej funkcie bez volania LLM.)

Z nameraných dát vyplýva, že fáza prepisu (*STT*) vykazuje relatívne stabilné hodnoty v rozmedzí od 1,9 do 2,7 sekundy, pričom mierny nárast je priamo úmerný dĺžke hlasového vstupu (napr. pri diktovaní).

Kritickým bodom z hľadiska odozvy sú **kreatívne úlohy**, kde celková latencia dosahuje až 8,7 sekundy. Detailnejšia analýza ukazuje, že až 60 % tohto času zaberá samotná exekúcia úlohy, čo je spôsobené generovaním veľkého počtu nových tokenov modelom LLM. Naopak, pri **jednoduchých a formátovacích príkazoch** je fáza exekúcie výrazne menšia, keďže sa spracúvajú lokálne v rámci rozhrania editora a nevyžadujú ďalšie volanie externých API.

Najvyššia miera latencie bola zaznamenaná pri kreatívnom príkaze „*Vygeneruj príbeh o Jankovi a Marienke*“, kde celkový čas spracovania dosiahol až 13,2s. Z toho fáza exekúcie (*Task*) zabrala 8,5s, čo je priamo úmerné rozsahu generovaného textu. Na opačnom konci bol zaznamenaný najrýchlejší pokus pri príkaze „*Zväčši písmo o dva body*“, ktorý systém spracoval za 2,4s.

Napriek tomu, že spracovanie komplexných úloh trvá viac ako 6 sekúnd, systém poskytuje používateľovi priebežnú informáciu o stave operácie, čím sa táto odozva stáva pre kreatívnu prácu prijateľnou.

6.3 Diskusia o limitoch a presnosti interpretácie príkazov

Výsledky testovania ukázali, že hoci systém vykazuje vysokú mieru úspešnosti pri bežných úlohách, existujú špecifické limity dané architektúrou a použitými modelmi.

Fonetické limity a vplyv STT Najvýraznejším limitom z hľadiska presnosti je závislosť systému od kvalite prepisu modelu *Whisper*. Pri testovaní sa opakovane potvrdilo, že skomolené a foneticky podobné slová môžu spôsobiť nesprávnu klasifikáciu v *Router Agente*.

Schopnosť systému porozumieť zámeru Pozitívnym zistením je schopnosť *Router Agenta* úspešne interpretovať príkazy aj v prípade, že prepis obsahuje drobné gramatické chyby alebo preklepy. Na rozdiel od klasických systémov založených na kľúčových slovách, náš systém dokázal identifikovať zámer používateľa na základe kontextu celej vety. Limity sa však prejavili pri extrémne dlhých a viaczložkových príkazoch, kde sa ojedinele stávalo, že systém jednu z čiastkových úloh vynechal.

Segmentácia textu a spoľahlivosť editácie V priebehu testovania sa potvrdilo, že pri úprave rozsiahlych textových blokov narastá riziko, že *Edit Agent* nevykoná editačnú úlohu precízne. Veľké jazykové modely vo všeobecnosti dosahujú vyššiu mieru spoľahlivosti pri práci s menším množstvom informácií v rámci jedného kontextového okna. Z tohto dôvodu je logika nášho systému navrhnutá tak, aby s dokumentom nepracovala ako s nedeliteľným celkom, ale aby ho segmentovala na jednotlivé odseky. Tento prístup výrazne znižuje pravdepodobnosť, že model pri editácii stratí pozornosť alebo neúmyselne pozmení časti textu, ktoré nemali byť predmetom úpravy.

Záver

Predložená bakalárska práca sa venovala návrhu a implementácii inovatívneho textového editora vo forme webovej aplikácie, ktorého jadrom je ovládanie prostredníctvom prirodzeného jazyka. Hlavným cieľom bolo odstrániť technickú bariéru medzi myšlienkou a jej zapísaním, čo sa podarilo realizovať transformáciou tradičného statického rozhrania na inteligentný, hlasom ovládaný systém.

Dosiahnuté ciele a prínosy

V rámci práce sme úspešne prepojili moderné webové technológie s pokročilými modelmi umelej inteligencie:

- **Agentická architektúra:** Implementovali sme koncept *Agentic AI*, kde systém nefunguje len ako pasívny nástroj, ale ako aktívny digitálny spoluautor. Využitie multi-agentového systému riadeného *Router Agentom* umožnilo dekompozíciu komplexných používateľských príkazov na konkrétne úlohy.
- **Technologická integrácia:** Aplikácia efektívne kombinuje knižnicu *React* pre dynamické zobrazovanie, framework *LangChain* pre riadenie logiky agentov a modely GPT-4o a *Whisper* od spoločnosti OpenAI pre spracovanie textu a reči.
- **Prirodzená interakcia:** Vďaka integrácii *VAD* sme dosiahli plynulé ovládanie bez nutnosti manuálneho spúšťania nahrávania, čím sa editor priblížil k prirodzenej ľudskej konverzácii.

Vyhodnotenie výsledkov

Verifikačná fáza potvrdila životaschopnosť navrhnutej architektúry. Testovanie na 97 scenároch preukázalo celkovú úspešnosť vykonania príkazov v slovenskom jazyku na úrovni 82,5 %. Analýza ukázala, že systém je vysoko robustný pri sémantickom chápaní dlhších a komplexnejších viet, zatiaľ čo hlavným limitom zostáva presnosť prepisu krátkych slovenských povelov modelom *Whisper*. Z hľadiska latencie systém reaguje v

prijateľnom čase, pričom jednoduché navigačné a formátovacie úkony sú spracované v priebehu 3 až 4 sekúnd.

Budúce smerovanie

Navrhnutý systém predstavuje funkčný základ pre ďalší rozvoj *AI-natívnych* editorov. V rámci budúcich vylepšení sa otvára priestor pre integráciu špecializovaných agentov a rozšírenie vstupných metód, ktoré by výrazne zvýšili úžitkovú hodnotu editora.

Kľúčovým funkčným rozšírením by bola možnosť priameho nahrávania externých textových súborov alebo PDF dokumentov, ktoré by systém následne spracoval a pripravil na ďalšiu hlasovú editáciu. Jedným z ďalších smerov je implementácia *Formatting Agent*, ktorý by umožnil pokročilú prácu so štruktúrou, ako je automatická transformácia textu na tabuľky alebo aplikácia komplexného *Markdown* formátovania. Multimediálny obsah dokumentu by mohol dopĺňať *Image Generation Agent*, schopný na základe kontextu textu vytvárať ilustrácie (napr. cez modely typu *DALL-E 3*) a automaticky ich vkladať do dokumentu.

Záverom možno konštatovať, že práca úspešne demonštruje potenciál veľkých jazykových modelov pri transformácii softvérových rozhraní. Výsledná aplikácia potvrdzuje, že ovládanie softvéru prirodzenou rečou už nie je len teoretickým konceptom, ale prakticky realizovateľnou cestou k zvyšovaniu produktivity a komfortu pri tvorbe digitálneho obsahu.

Literatúra

- [1] Sanchit Aggarwal et al. Modern web-development using reactjs. *International Journal of Recent Research Aspects*, 5(1):133–137, 2018.
- [2] Pablo Becker, Guido Tebes, Denis Peppino, and Luis Antonio Olsina Santos. Applying an improving strategy that embeds functional and non-functional requirements concepts. *Journal of Computer Science & Technology*, 19, 2019.
- [3] Deepgram Inc. Deepgram. <https://deepgram.com/>, 2026. [Online; citované 2026-05-09].
- [4] Anisha Gajbahar. Ai productivity tools: A study on notion.ai. *Authorea Preprints*, júl 2025.
- [5] Marijn Haverbeke. *Eloquent javascript: A modern introduction to programming*. No Starch Press, 2018.
- [6] LangChain Inc. Langchain.js: Overview and documentation. <https://docs.langchain.com/oss/javascript/langchain/overview>, 2026. Online; citované 13. apríla 2026].
- [7] Colin McDonnell. Zod: Typescript-first schema declaration with static type inference. <https://zod.dev/>, 2024. [Online; citované 9. mája 2026].
- [8] Meta Platforms, Inc. Virtual dom and internals. <https://legacy.reactjs.org/docs/faq-internals.html>, 2024. [Online; citované 9. mája 2026].
- [9] Meta Platforms, Inc. React. <https://react.dev/>, 2026. [Online; citované 9. mája 2026].
- [10] Notion Labs, Inc. Notion ai. <https://www.notion.com/>, 2026. [Online; citované 8. mája 2026].
- [11] OpenAI. Whisper. <https://openai.com/index/whisper/>, 2022. [Online; citované 9. mája 2026].

- [12] OpenAI. Openai api documentation: Gpt-4o model capabilities. <https://developers.openai.com/api/docs/models/gpt-4o>, 2026. [Online; citované 2026-04-13].
- [13] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [14] Christopher Rickard. Vad - voice activity detection in the browser. <https://github.com/ricky0123/vad>, 2024. [Online; citované 8. mája 2026].
- [15] Sherry Ruan, Jacob O Wobbrock, Kenny Liou, Andrew Ng, and James Landay. Speech is 3x faster than typing for english and mandarin text entry on mobile devices. *arXiv preprint arXiv:1608.07323*, 2016.
- [16] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, 4th edition, 2020.
- [17] Silero Team. Silero vad. <https://github.com/snakers4/silero-vad>, 2024. [Online; citované 2026-05-09].
- [18] S. Sonkamble. Microsoft copilot: The future of ai-powered productivity in the workplace. *International Journal of Innovative Research in Technology (IJIRT)*, 12(1):2778–2782, 2025.
- [19] Voicy Team. Voicy: Voice-powered ai text editor. <https://usevoicy.com/>, 2024. Accessed: 2026-03-08.
- [20] Boshi Wang, Xiang Yue, and Huan Sun. Can chatgpt defend its belief in truth? evaluating llm reasoning via debate. In *Findings of the association for computational linguistics: EMNLP 2023*, pages 11865–11881, 2023.
- [21] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [22] Wispr AI. Wispr flow. <https://wisprflow.ai/>, 2026. [Online; citované 8. mája 2026].
- [23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.

- [24] Xiao-Lei Zhang and Ji Wu. Deep belief networks based voice activity detection. *IEEE Transactions on Audio, Speech, and Language Processing*, 21(4):697–710, 2013.

Príloha A: Obsah elektronickej prílohy

Elektronická príloha k bakalárskej práci obsahuje kompletne digitálne podklady, zdrojové kódy a dokumentáciu k vytvorenému systému.

Zdrojový kód: Kompletný repozitár projektu je dostupný v priloženom archíve a zároveň zverejnený na platforme GitHub: <https://github.com/Tomas-Telek/Prirodzenym-jazykom-ovladany-editor-vo-forme-webovej-aplikacie>

Online aplikácia: Pre účely okamžitého otestovania bez nutnosti lokálnej inštalácie je systém dostupný na adrese: <https://prirodzenym-jazykom-ovladany-editor.vercel.app>

Konfigurácia a prístup: Aplikácia pre svoju plnohodnotnú funkčnosť (spracovanie textu a hlasových príkazov) vyžaduje vlastný API kľúč od spoločnosti OpenAI. Tento kľúč je možné získať po registrácii na platforme <https://platform.openai.com/api-keys>. Po získaní kľúča je potrebné ho vložiť do konfiguračného poľa priamo v rozhraní aplikácie.